

ТЕМА:

Исследование процесса обучения модели с архитектурой PilgrimAttnRes.

Поиск формата обучения для достижения минимального времени обучения с получением процента найденных решений более 90.

ЗАДАЧИ:

1) Проанализировать таблицу из статьи

[HTTP://www.overleaf.com/project/69fdf2dcddacfe8cdc52df9d](http://www.overleaf.com/project/69fdf2dcddacfe8cdc52df9d).

Полная таблица:

<https://www.overleaf.com/project/69fdf2dcddacfe8cdc52df9d>

Ссылка на проект caylepy-rw-modelbaselines-megaminx на Kaggle:

<https://www.kaggle.com/code/alexandervc/caylepy-rw-modelbaselines-megaminx?>

2) Внести предложения по возможности уменьшения времени тренировки моделей при сохранении результата (%solv > 90) на основе исследования работы слоев модели.

ВВЕДЕНИЕ.

Краткое описание и анализ таблицы результатов из статьи, уточнение задачи с учётом выводов по таблице.

С базовыми параметрами конфигурации можно ознакомиться непосредственно в статье. В данной работе будет подробнее рассмотрена лишь небольшая часть исследования, касающаяся некоторых аспектов архитектуры и параметров тренировки. В статье же можно посмотреть результаты по тюнингу таких параметров, как размер батча (1к - 40к), подбор loss function, большое число эпох (1к - 60к), Beam Size, Random walk length, ...

ОПИСАНИЕ АРХИТЕКТУРЫ PilgrimAttnRes (этот вариант используется в данной работе):

(input_layer): Linear - входной линейный слой (описаны размеры out_features от 32 до 4096), сжимает количество входных значений до заданного размера out_features.

(bn1): BatchNorm1d - батчнорм слой, нормализует активации входного линейного слоя (input_layer) по мини-батчам.

(relu): ReLU() - слой функции активации - добавляет нелинейность, помогает выловить-выучить сложные (нелинейные) паттерны и закономерности в данных

(dropout): Dropout - дропаут слой (пока не используется)

(hidden_layer): Linear - скрытый линейный слой (описаны размеры out_features от 32 до 2048) - сжимает представление от размера входного слоя до заданного out_features.

(bn2): BatchNorm1d - батчнорм слой, нормализует активации скрытого линейного слоя (hidden_layer) по мини-батчам.

(residual_blocks): ModuleList - блоки из повторяющейся комбинации слоёв. Блоки связаны через skip connection, что позволяет не изучать новое представление, а корректировать представление, поступившее на вход. Количество блоков задано через nrd (number of residual blocks), описаны nrd=1,2,4,8,16. Все слои одинакового размера (как у скрытого слоя). Лучшие результаты получены для моделей с nrd=4.

(0-...): nrd x ResidualBlock:

(fc1): Linear
(bn1): BatchNorm1d
(relu): ReLU
(dropout): Dropout
(fc2): Linear
(bn2): BatchNorm1d

(output_layer): Linear - выходной линейный слой (target - одно значение, regression output (используется MSELoss)).

АНАЛИЗ ТАБЛИЦЫ РЕЗУЛЬТАТОВ ИЗ СТАТЬИ.

В статье приведена таблица результатов запуска beam search для модели PilgrimAttnRes обученной в разных вариантах архитектуры (размеры слоев (L1 - входной слой, L2 - скрытый слой), количество residual blocks - nrd (number of residual blocks) --> L1 - L2 x nrd), при разных параметрах (LR - learning rate, batch_size - размер батча, epochs - число эпох обучения). С разным Scheduler - постоянный LR и LR, изменяющийся по косинусу.

Ниже приводится часть этой таблицы.

Описание таблицы:

Результат запуска beam search:

%Solv - процент найденных решений
Len - средняя длина пути
Std - стандартное отклонение

Также приведены:

Loss - val_loss mean average (window=50).
Средний val_loss по последним 50 эпохам.
Model - время (продолжительность) тренировки
Beam - время (продолжительность) beam search

Train (тут не показан) - размер тренировочного датасета (определяется конфигурацией rw_width, rw_length и числом эпох обучения)*

Size (тут не показан) - размер модели (зависит от размера слоев и числа блоков)

* - важной особенностью тренировки и валидации является то, что каждую эпоху модель тренируется и валидируется на **новом наборе данных**. Поэтому, чем больше эпох обучения, тем больше тренировочный набор.

Таблица 1: Влияние размера входного слоя

index	Architecture	%Solv	Len	Std	Loss	Epochs	LR	Model	Beam
0	Layer 1								
1	ResMLP 32-32x4	0	-	-	63.5	1000	1e-3	25m	6h26m
2	ResMLP 64-64x4	33	676	204	61.6	1000	1e-3	25m	5h52m
3	ResMLP 128-128x4	70	551	166	59.4	1000	1e-3	28m	4h40m
4	ResMLP 256-256x4	92	392	120	55	1000	1e-3	32m	3h10m
5	ResMLP 512-256x4	94	337	110	55.7	1000	1e-3	41m	2h55m
6	ResMLP 1024-256x4	99	261	61	52	1000	1e-3	1h15m	2.5h
7	ResMLP 2048-256x4	95	254	72.5	51.5	1000	1e-3	1h51m	3.5h
8	ResMLP 4096-256x4	99	234	58.5	50.7	1000	1e-3	3h20m	4h20
9	ResMLP 8192-256x4	99	221	43	50.1	1000	1e-3	7h5m	8h47m

Из Таблицы 1 очевидно, что чем больше размер L1 (больше learned features - полнее представление), тем лучше %solv, короче путь и меньше std. Размер L1 - ключевой параметр архитектуры, чем он больше, тем лучше. Меньше, чем 256 не стоит рассматривать.

Таблица 2: Влияние размера скрытого слоя.

index	Architecture	%Solv	Len	Std	Loss	Epochs	LR	Model	Beam
10	256-L2								
11	ResMLP 256-64x4	90	430	111	57.7	1000	1e-3	30m	3h26m
12	ResMLP 256-128x4	95	359	95	57.2	1000	1e-3	30m	2h43m
13	ResMLP 256-256x4	92	392	120	55	1000	1e-3	32m	3h10m

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
14	ResMLP 256-512x4	90	410	14.4	58.3	1000	1e-3	40m	3h26m
15	ResMLP 256-1024x4	57	413	12.4	62.6	1000	1e-3	1h8m	5h47m
16	ResMLP 256-2048x4	82	489	15.1	62	1000	1e-3	2h46m	7h34m
17	ResMLP 512-256x4	94	337	11.0	55.7	1000	1e-3	41m	2h55m

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
18	1024-L2								
19	ResMLP 1024-64x4	96	276	80	53.2	1000	1e-3	1h8m	2h49m
20	ResMLP 1024-128x4	95	264	10.0	52.7	1000	1e-3	1h9m	2h46m
21	ResMLP 1024-256x4	99	261	61	52	1000	1e-3	1h15m	2.5h
22	ResMLP 1024-512x4	99	278	95	53.6	1000	1e-3	1h20m	2h43m
23	ResMLP 1024-1024x4	99	272	73	54.4	1000	1e-3	1h53m	3h7m

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
24	higher-L2								
25	ResMLP 2048-64x4	98	261	86	52.0	1000	0.001	1h46m	3h18m
26	repeat	93	265	57	51.3	1000	0.001	1h46m	3h45m
27	ResMLP 2048-128x4	96	254	105	51.4	1000	0.001	1h48m	3h25m
28	repeat	98	231	48	51.2	1000	0.001	1h48m	2h56m
29	ResMLP 2048-256x4	95	254	72.5	51.5	1000	1e-3	1h51m	3.5h
30	ResMLP 2048-512x4	98	252	96	51.9	1000	1e-3	2h1m	3h17m
31	ResMLP 2048-1024x4	95	293	77	52.8	1000	1e-3	2h32m	4h28m
32	ResMLP 2048-2048x4	88	242	48	53	1000	1e-3	4h17m	6h17m
33	ResMLP 4096-128x4	90	258	74	50.7	1000	0.001	3h16m	5h59m
34	repeat	98	230	45	50.5	1000	0.001	3h17m	4h29m
35	ResMLP 4096-256x4	99	234	58.5	50.7	1000	1e-3	3h20m	4h20
36	ResMLP 4096-512x4	99	244	78.2	51.1	1000	1e-3	3h32m	4h45
37	ResMLP 4096-1024x4	99	242	85	51.7	1000	0.00	4h6m	5h4m

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
							1		

Из Таблицы 2 видно, что размер L2 имеет некоторое оптимальное значение для заданного L1. И это оптимальное значение тем больше, чем больше L1. Для 256 - это 128, для 1024 - 256, и т.д. При этом влияние L2 не такое сильное, как L1 - изменение размера L2 слабее влияет на результат.

Таблица 3: Влияние числа residual blocks (Depth)

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
0	Depth								
1	ResMLP 256-128x1	98	375	109	57.4	1000	1e-3	26m	2h37m
2	ResMLP 256-128x2	99	375	121	57.0	1000	1e-3	27m	2h36m
3	ResMLP 256-128x4	95	359	95	57.2	1000	1e-3	30m	2h43m
4	ResMLP 256-128x8	96	375	121	57.0	1000	1e-3	38m	2h52m
5	ResMLP 256-128x16	97	422	158	58.2	1000	1e-3	59m	3h39m
6	ResMLP 1024-256x1	97	265	53	53.6	1000	1e-3	1h7m	2h37m
7	ResMLP 1024-256x2	100	257	68	53.3	1000	1e-3	1h9m	2h22m
8	ResMLP 1024-256x4	99	261	61	52	1000	1e-3	1h15m	2.5h
9	ResMLP 1024-256x8	97	294	108	53.4	1000	1e-3	1h21m	3h6m
10	ResMLP 2048-256x1	99	239	63	52.5	1000	1e-3	1h46m	2h59m
11	ResMLP 2048-256x2	96	266	78	51.3	1000	1e-3	1h47m	3h30m
12	ResMLP 2048-256x4	95	254	72.5	51.5	1000	1e-3	1h51m	3.5h
13	ResMLP 2048-256x8	98	235	61	51.8	1000	1e-3	1h58m	3h7m
14	repeat	98	246	68	51.4	1000	1e-3	1h58m	3h16m
15	ResMLP 4096-256x1	100	242	60	50.7	1000	0.001	3h15m	4h22m
16	repeat	99	245	94	50.2	1000	0.001	3h15m	4h34m
17	ResMLP 4096-256x2	97	239	71	50.7	1000	0.001	3h17m	4h44m
18	repeat	98	228	47	50.7	1000	0.001	3h17m	4h23m
19	ResMLP 4096-256x4	95	235	60	50.7	1000	0.001	3h20m	5h0m
20	repeat	93	234	49	50.4	1000	0.001	3h20m	5h15m
21	ResMLP 4096-256x8	95	226	99	50.9	1000	0.001	3h27m	4h55m

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
22	repeat	96	232	52	50.4	1000	0.001	3h27m	4h57m
23	ResMLP 4096-256x16	100	220	46	50.9	1000	0.001	3h40m	4h12m
24	repeat	93	228	54	50.7	1000	0.001	3h39m	5h25m

Из Таблицы 3 видно, что увеличение числа блоков при небольших размерах входного слоя (256, 1024) приводит к накоплению шума при $nrd > 4$, а при значениях до 4 - выигрыш не очевиден. Чем больше размер входного слоя (и соответственно размер скрытого слоя и ResidualBlocks), тем стабильнее итоговый результат - увеличение числа блоков повышает точность результата и сокращает длину найденного пути (до порогового nrd , который тем больше, чем больше размер входного слоя).

Также видно, что улучшение, которое выигрывается за счет увеличения числа блоков, на порядок меньше, чем за счет увеличения размера входного слоя. Тем не менее, количество числа блоков слабо влияет на время тренировки модели и время beam search, поэтому является хорошим улучшением при условии достаточного размера входного слоя. Однако тренировка моделей с большой глубиной требует эффективного использования вычислительных ресурсов. При запуске обучения в Google Colab оказалось, что увеличение числа блоков приводит к сокращению разрешенного времени вычислений (colab принудительно прерывает сессию). Даже для минимального размера (256-128x) работа с $nrd=4$ в Google Colab оказывается сложной, можно делать запуски только на небольшое число эпох.

Таблица 4: Влияние Scheduler.

index	Architecture	%Solved	Length	Std	Loss	Epochs	LR	Model	Beam
0	ResMLP 256-128x1 With Scheduler								
1	Cosine 1e-2/3e-4	95	311	141	48.3	1000	1e-2	26m	2h20m
2	repeat	100	249	50	47.8	1000	1e-2	26m	1h40m
3	Cosine 4e-3/1e-4	98	258	58	48.2	1000	4e-3	26m	1h55m
4	repeat	100	235	44	47.8	1000	4e-3	26m	1h34m

Из Таблицы 4 видно, что переменный LR существенно улучшает результат (для ResMLP 256-128x1 -- с длины пути 375 при постоянном LR до примерно 250 при переменном). Это справедливо и для других значений размеров модели, но тут эти результаты не приводим.

Выводы по не приведенным в этой работе данным таблицы:

1) размер батча = 1000 оптимален (достаточно большой).

2) LR (от $1e-3$ до $1e-5$) - уже с $LR=1e-4$ небольшая модель (входной слой 256, $nrd=1$) недообучается, но вполне может достичь приемлемого %solv при большем размере модели (входной слой 1024, $nrd=4$).

Постоянные большие LR ($1e-1$, $1e-2$) не рассмотрены.

3) Увеличение числа эпох с 1000 до 10000 уменьшает длину пути и std, но нет очевидного влияния на %solv. Дальнейшее увеличение числа эпох (не смотря на уменьшение лосса) может оканчиваться провалом beam search.

4) Val_loss_MA50 - не является однозначной метрикой предсказывающей результат beam_serch. Поиск факторов, которые совместно с Val_loss_MA50 коррелируют с результатом, остается актуальной задачей.

Описание задачи с учетом анализа таблицы (уделить внимание нерассмотренным аспектам):

С точки зрения уменьшения времени тренировки можно изучить:

- уменьшение числа эпох;
- влияние использования best_epoch для beam_search;
- возможность использования пороговых значений лосса для ранней остановки;
- влияние изменения LR (в базовой таблице лучший результат достигнут для переменного LR по косинусу)) - задача не улучшить имеющиеся в таблице результаты, а объяснить их;

В рамках этой работы посмотрим подробнее на обучение при различных LR, в том числе переменных (пошаговое снижение), и разных LR для разных слоев. Посмотрим, как изменяются веса и градиенты на каждом слое по мере тренировки, какой вклад в результат дает каждый из слоев.

Для достижения поставленных целей добавим к лоссам несколько метрик, характеризующих состояние системы и описывающих скорости процессов как по обученной модели в целом, так и в разрезе слоев.

Существенно перепишем код тренировки и обработки результатов: перейдем от последовательного применения функций к ООП. Такой подход позволяет удобно тренировать в одном ноутбуке несколько кривых (curve - объект, хранящий историю обучения), например, имеющих частично общую историю. Появляется возможность запустить эксперимент целиком в одном ноутбуке, отфильтровать чекпоинты для beam search, получить таблицу результатов эксперимента.

ОПИСАНИЕ КЛАССОВ ДЛЯ ТРЕНИРОВКИ МОДЕЛЕЙ

Ранее код представлял собой последовательную цепочку запуска функций или просто блоков кода. Словари конфиг (cfg) и результаты (paper_dict) относились к 1 запуску обучения модели. Функция train_model содержала в себе тренировку, валидацию, сохранение в словарь кривых обучения train_loss, val_loss. Модель последовательно инициировалась, тренировалась, сохранялся чекпоинт. На этой модели проводился error and prediction analysis; запускался beam_search.

Сейчас код адаптирован следующим образом:

Конфиг (cfg) и результаты (paper_dict) - это часть, общая для всех экспериментов в тетрадке.

Код работы с моделью и результатами разбит на 4 класса:

Curve
CheckpointInfo
Trainer
ExperimentManager

Curve - класс, хранящий историю обучения (метрики, чекпоинты). В нем задаются параметры обучения, которые не вошли в конфиг, т.е. относящиеся к конкретной кривой, конкретному запуску этапа обучения (обучение может быть поэтапным). При помощи этого класса автоматически можно дать кривой имя, которое используется для формирования названий файлов. Есть функции работы с кривой - загрузить, сохранить, клонировать, extend (чтобы продолжить обучать кривую), ...

CheckpointInfo - класс, хранящий информацию о конкретном чекпоинте. Если Curve - это расширение конфига, то CheckpointInfo - расширение paper_dict.

Trainer (или TrainerLight, если не нужно считать дополнительные метрики и сохранять лишние чекпоинты) - содержит функции для тренировки модели: тренировка эпохи, валидация эпохи; функции сохранения чекпоинтов и определения условий сохранения. Однако, вычисление метрик - эти функции не включены в класс.

ExperimentManager - принимает на вход кривую, инициализирует модель, оптимайзер, Trainer, подгружает чекпоинт - готовит модель к тренировке. Тренировка модели запускается через менеджер. Если is_light=True, то менеджер запустит быструю версию тренировки.

Процесс в итоге выглядит следующим образом:

1. Задаем параметры тренировки через инициализацию класса Curve.
2. Готовимся к тренировке через инициализацию класса ExperimentManager для конкретного объекта curve.
3. Запускаем тренировку - manager.train(curve).

Процесс тренировки можно организовать так, что в памяти будет находиться только одна модель (в менеджере) и ее кривая (кривых можно несколько, они занимают мало). Кривые и модели можно подгружать в циклах.

Ранее: «Model error and prediction analysis», «Predict on samples», «Beam search» - были ячейками с кодом, который применялся к текущей модели. Теперь эти ячейки превращены в функции. Нужная модель подгружается из чекпоинта в цикле. Написанные ранее классы для этого не используются. Для запуска этих частей кода нужны имена файлов чекпоинтов.

`paper_dict` служит основой для словаря `paper_by_name = {ckpt_file_name: paper_dict}`, который по названию файла чекпоинта содержит соответствующий `paper_dict`.

Пока что этот код не интегрирован в общую систему заполнения глобальной таблицы результатов и выполнения множественных сабмитов.

Поскольку относительно исходной версии выполняется много сторонних вычислений (метрики) - время тренировки занимает заметно больше времени. Класс `Trainer` хорош для исследовательских целей. Если нужен только чекпоинт для Beam search, лучше использовать класс `TrainerLight`.

КРАТКОЕ ОПИСАНИЕ ДОПОЛНИТЕЛЬНЫХ МЕТРИК ДЛЯ АНАЛИЗА ПРОЦЕССА ОБУЧЕНИЯ

Net_metrics - метрики, применимые к состоянию модели целиком.

`epoch_displacement` - абсолютное значение изменения параметров за эпоху.

`displacement = current_params - previous_params`.

Большие значения говорят о том, что оптимайзер дает большие обновления.

`relative_parameter_displacement` - отношение `displacement` к `previous_params` - указывает на масштаб изменений по отношению к предыдущему состоянию. По смыслу - убывающая величина. Параметры как правило растут, доля обновления по отношению к величине параметра уменьшается.

`cosine_update` - `cosine_similarity`, согласованность между апдейтами.

`cosine_update = cosine_similarity(displacement, previous_displacement)`

`update_variance` - насколько большой разброс в величине обновлений в некотором интервале.

`validation_improvement` - изменение в величине `val_loss` за эпоху

`validation_improvement = prev_val_loss - val_loss`

`improvement_per_displacement` - нормализованное отношение изменения `val_loss` к изменению параметров

$improvement_per_displacement = validation_improvement / displacement$

Layer metrics - метрики, которые можно посмотреть по уровням. Причем уровнем может быть и вся архитектура целиком, или ResidualBlocks как блок, или блок из ResidualBlocks по отдельности, или каждый уровень по отдельности.

grad_norm - величина градиента, отражает реакцию на запрос лосса к изменениям. Если на одном уровне градиенты больше в 10 раз, чем на другом - значит лосс хочет обновить этот уровень в 10 раз сильнее.

$grad_norm = \sqrt{\sum(p.grad.norm()^2)}$

weight_norm - размер весов. Большой размер слоя изначально порождает большие веса. Но при одинаковом размере уровней позволяет сравнивать масштаб параметров. Динамика изменений весов дает нам такую метрику как displacement.

$weight_norm = \sqrt{\sum(p.data.norm().item()^2)}$

grad_weight_ratio - отражает силу запроса на изменение обновления.

$grad_weight_ratio = grad_norm / weight_norm$

adam_update_norm - фактически произошедшее обновление. Величина приблизительно отражает отношение среднего градиента к обычной величине градиента (грубая формулировка для $m/\sqrt{v}$).

$adam_update_norm = -lr * m / (\sqrt{v} + \epsilon)$ - где m, v - bias-corrected первое и второе приближение моментов (Exponential Moving Average градиентов). Поэтому на большие градиенты Адам не обязательно будет давать большое обновление, т.к. выучит, что для этого уровня, например, градиенты всегда большие. Чем больше lr (learning rate), тем больше обновление. ϵ - маленькая величина, чтобы избежать деления на ноль.

update_weight_ratio - характеризует скорость обновления, лучший индикатор активности на уровне.

$update_weight_ratio = adam_update_norm / weight_norm$

Т.е. запрос на изменение размера обновления может быть grad_weight_ratio, но фактически изменяется обновление со скоростью update_weight_ratio. Лосс через градиенты просит обновить уровень сильнее, а Адам пересчитывает обновление исходя и из других соображений тоже.

Интерпретация значений update_weight_ratio для описания состояния обучения:

- >1e-1 huge updates
- 1e-2 aggressive learning
- 1e-3 healthy
- 1e-4 slow refinement
- 1e-5 nearly frozen
- <1e-6 dead

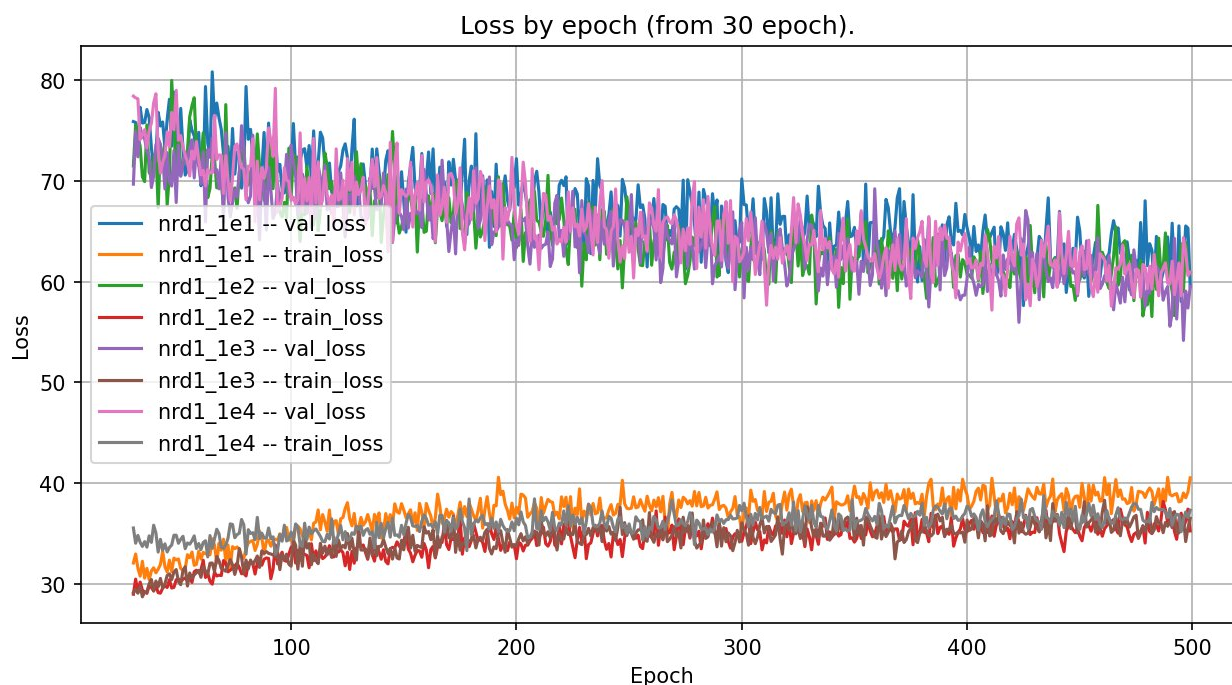
ИССЛЕДОВАНИЕ

Кривые обучения при постоянном LR для одной и той же архитектуры выглядят очень похоже. После 100 эпохи все они находятся очень близко. Трэйн и валидация, каждый образует плотный пучок. При этом результаты beam_search на 1000 эпохах для этих кривых будут отличаться (отличие трудно оценить из-за зашумленности данных)

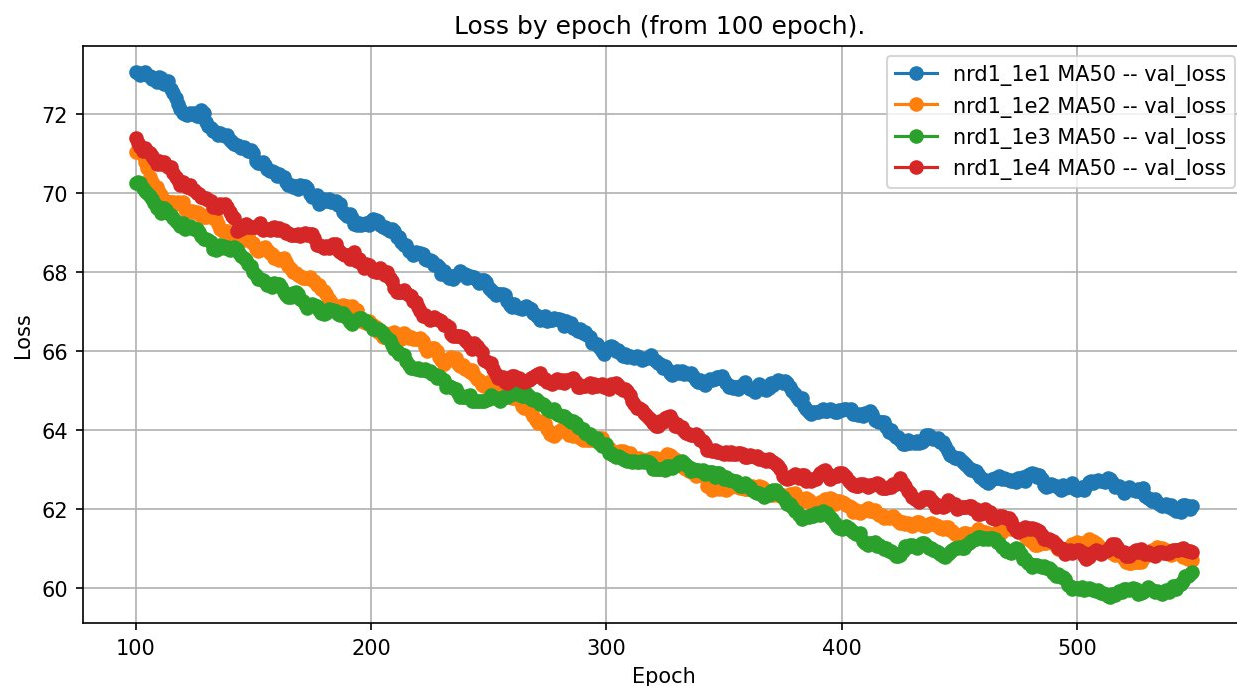
Результаты из статьи, и запуски в рамках этой работы (в скобках номер запуска на kaggle)

index	Architecture	%Solv	Len	Std	Loss	Epochs	LR	Model	Beam
0	ResMLP 256-128x1	98	375	109	57.4	1000	1e-3	26m	2h37m
1	ResMLP 256-128x1 (157)	98	329	74	57.0	1000	1e-3	26m	2h19m
2	ResMLP 256-128x1 (162)	97	358	110	57.0	1000	4e-3	26m	2h35m
3	ResMLP 256-128x1	99	316	78	58.9	1000	4e-3	26m	2h11m
4	repeat	96	365	115	57.3	1000	4e-3	26m	2h40m
5	repeat	100	324	89	57.2	1000	4e-3	26m	2h21m
6	ResMLP 256-128x1	98	340	88	57.3	1000	1e-2	26m	2h24m
7	ResMLP 256-128x1 (233)	99	346	98	57.6	924 best	1e-2	-	2h22m

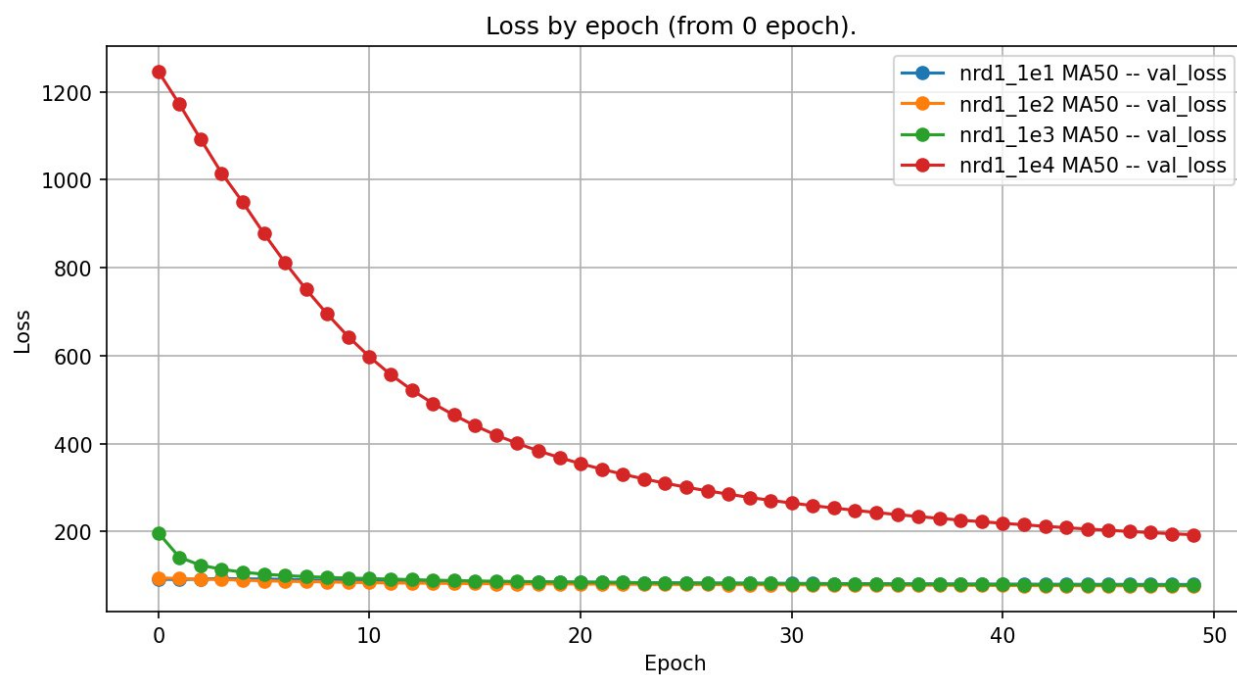
Кривые обучения ResMLP 256-128x1 при различных постоянных LR.



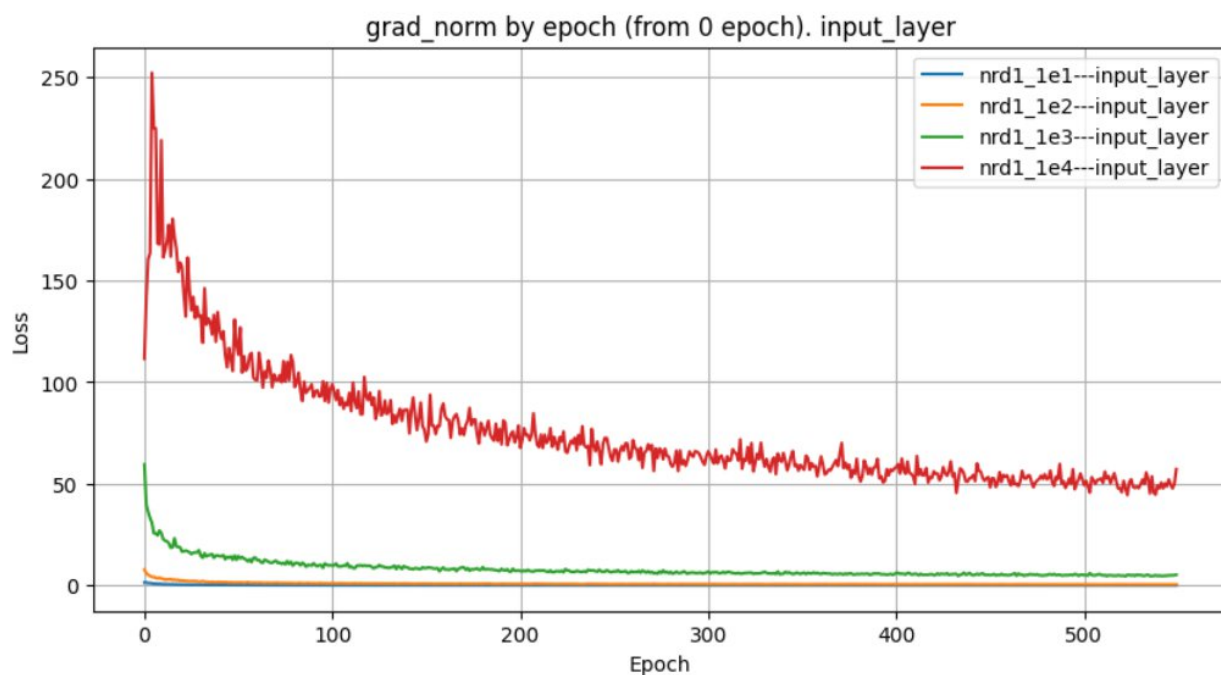
Можно видеть, что при LR=1e-1 модель переобучается. Валидационный лосс вскоре после первых эпох всегда хуже, чем при более низких LR. Но в целом кривые держатся рядом.



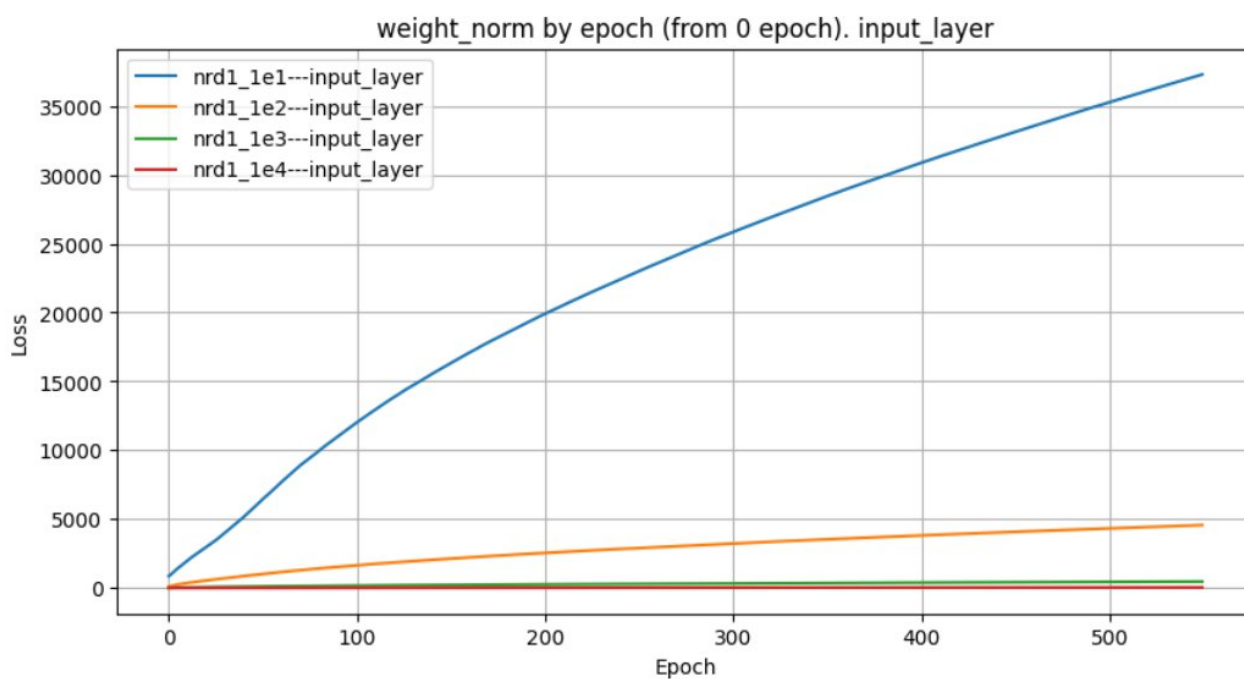
На первых эпоках порядок величины лоссов для разных LR сильно отличается. Малые LR стартуют с больших лоссов.



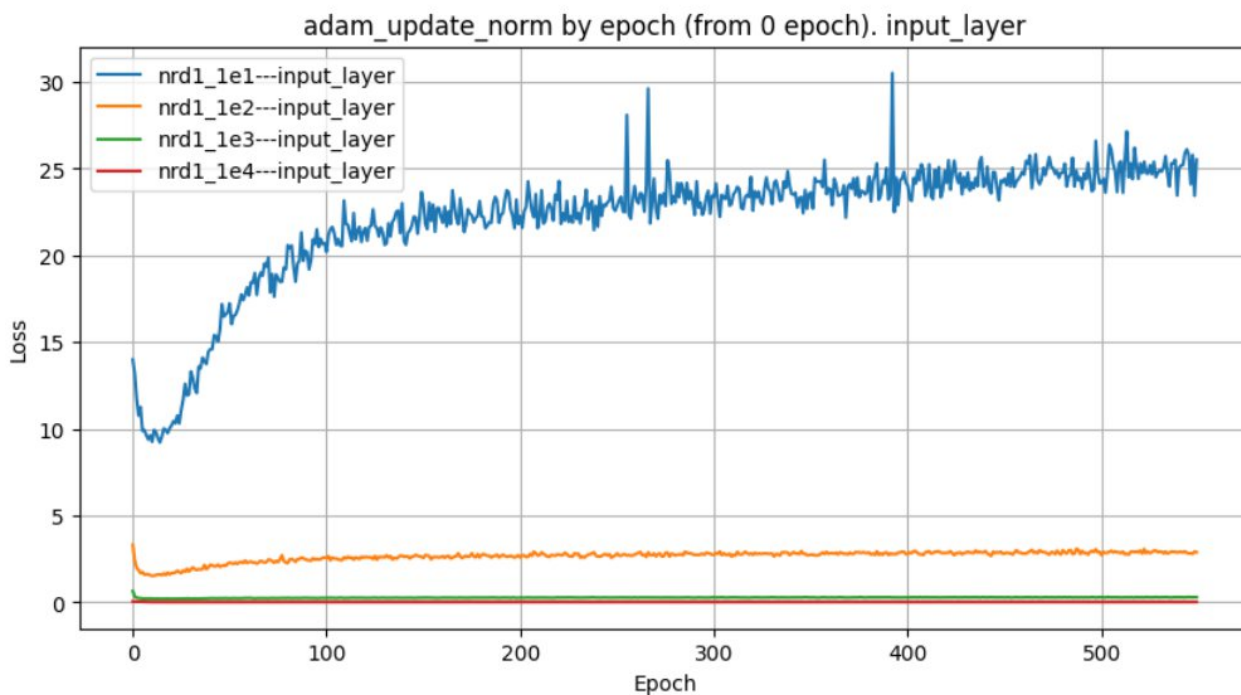
Градиенты на входном уровне большие при тренировке с малым LR



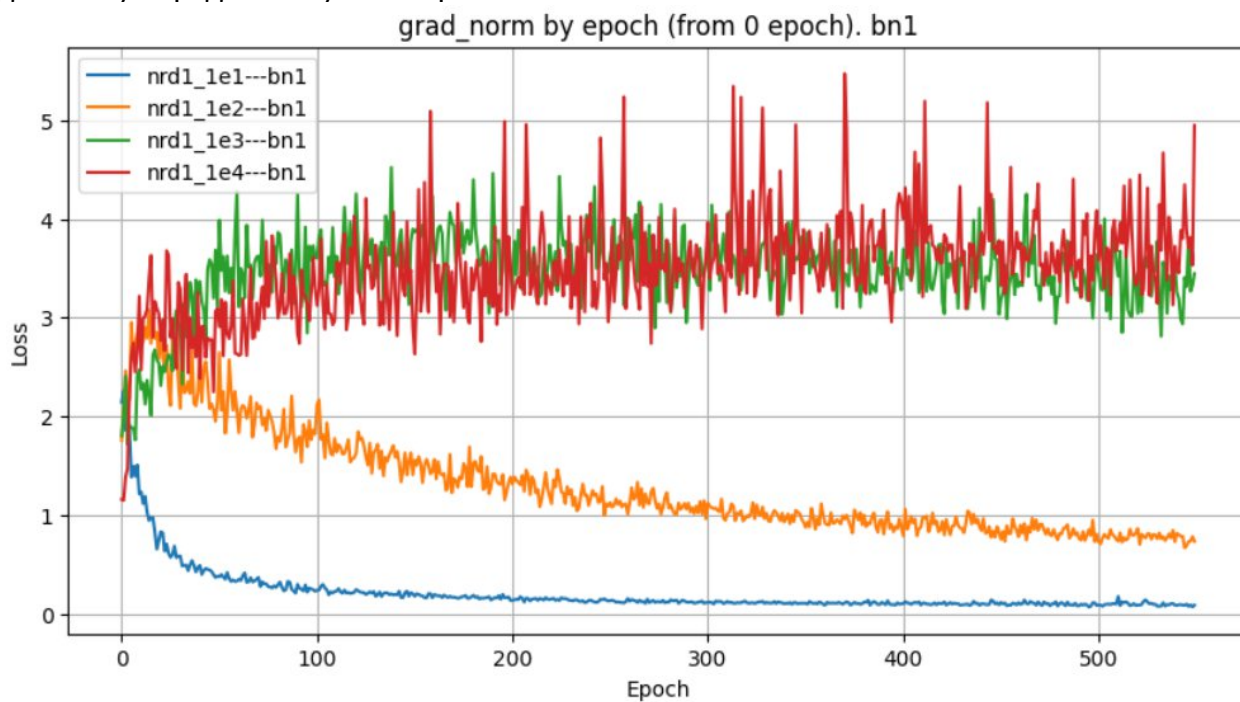
При LR = $1e-1$ - $1e-3$ веса на входном слое очень быстро растут по мере обучения, а лосс активно падает только на первых эпохах.



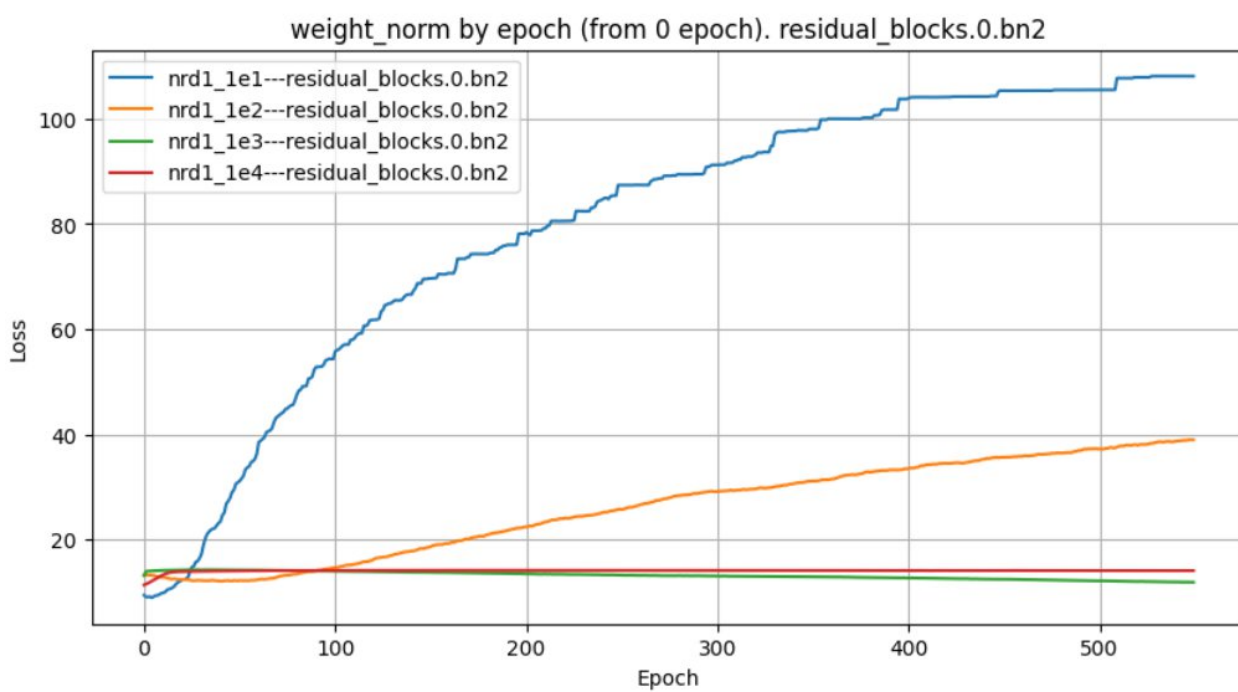
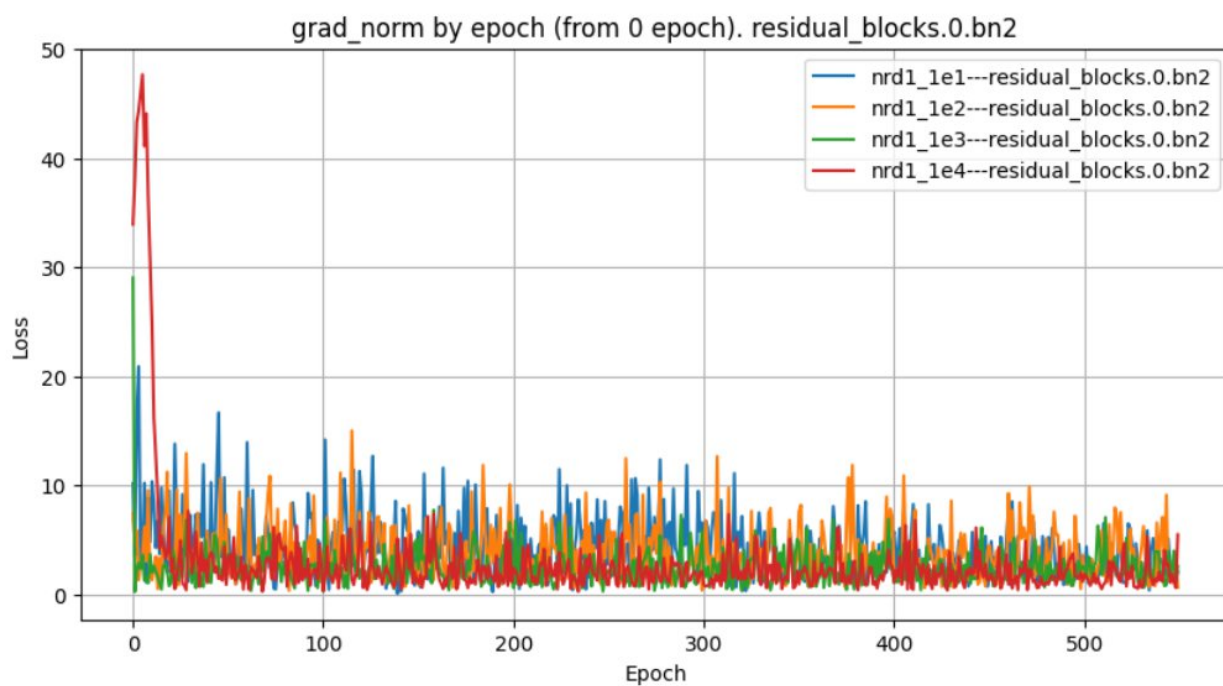
На кривой adam_update для входного слоя очень ярко видна «ямка» - признак начала режима инфляции весов.

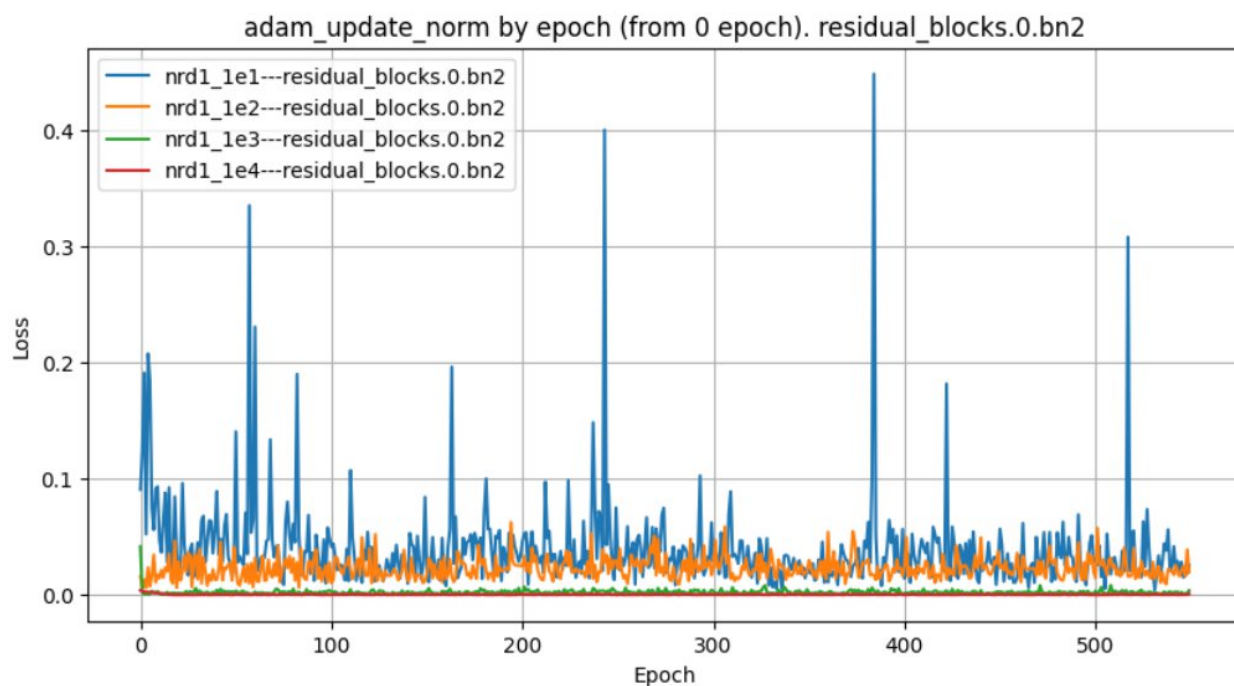


Градиенты на входном слое со временем уменьшаются - это динамика для всех LR. Градиенты на батчнорм слое имеют разную динамику в зависимости от LR. Возможно, это объясняется тем, что при больших LR соответствующий линейный слой находится в инфляционном режиме, а при малых - нет. Разные режимы линейного слоя дают разную динамику в градиентах у батчнорм слоев.

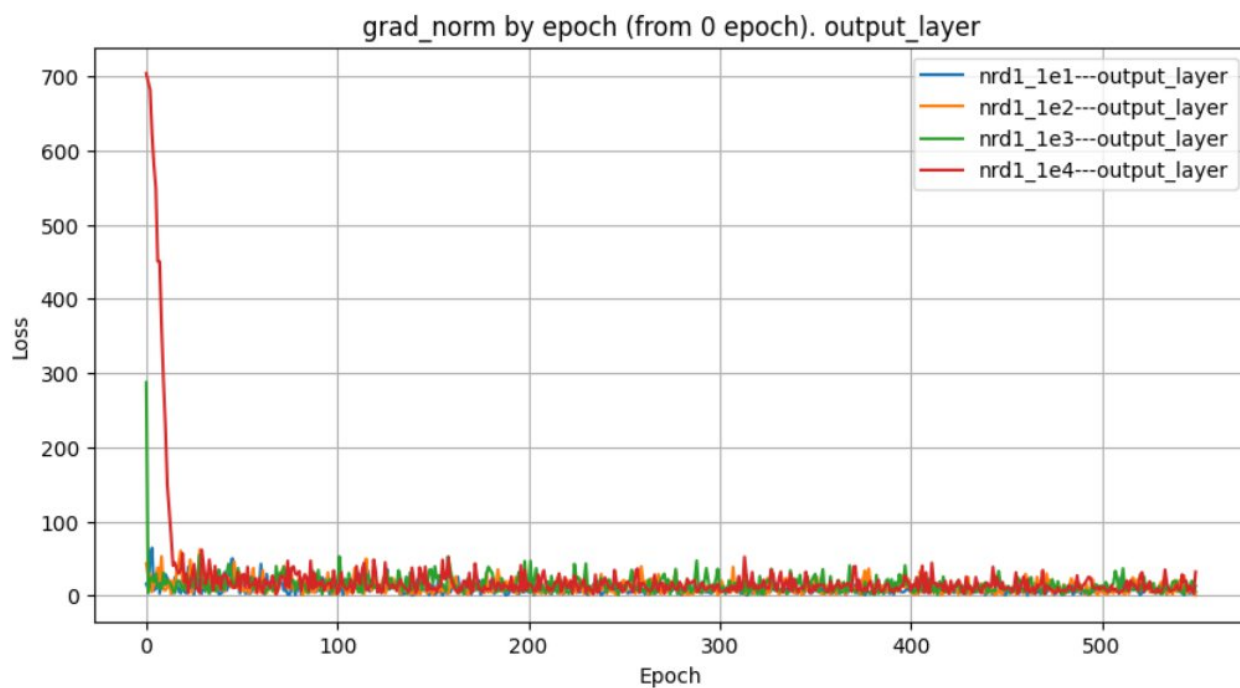


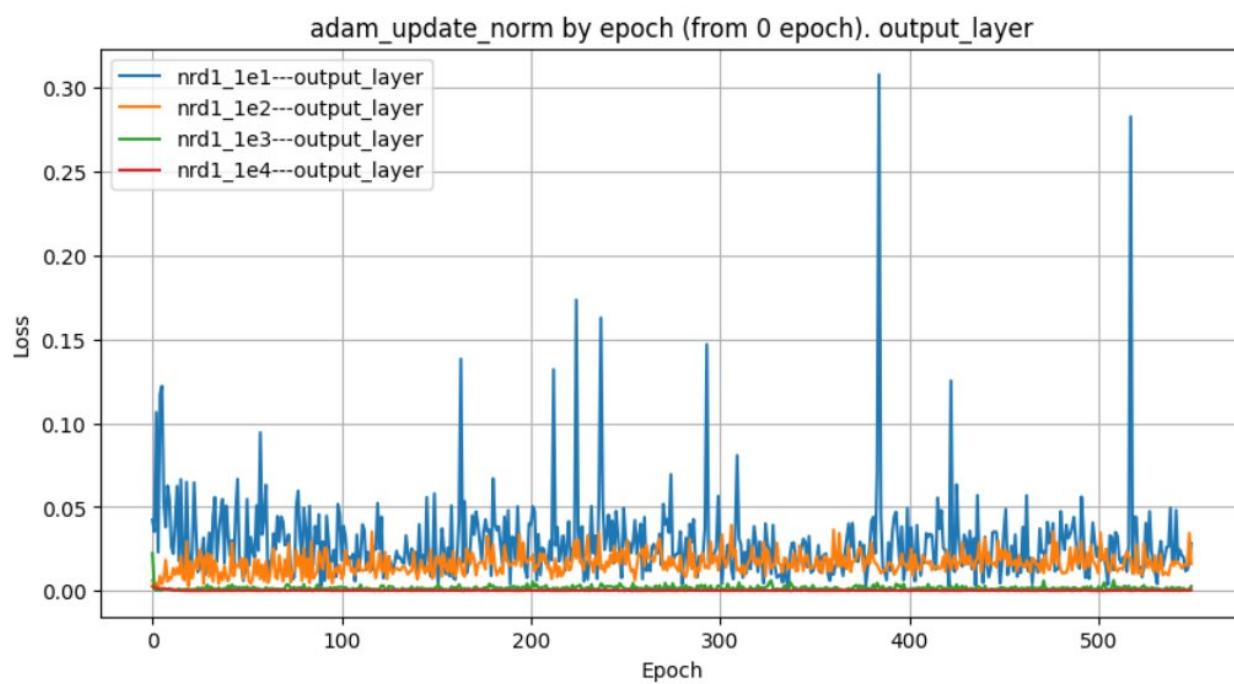
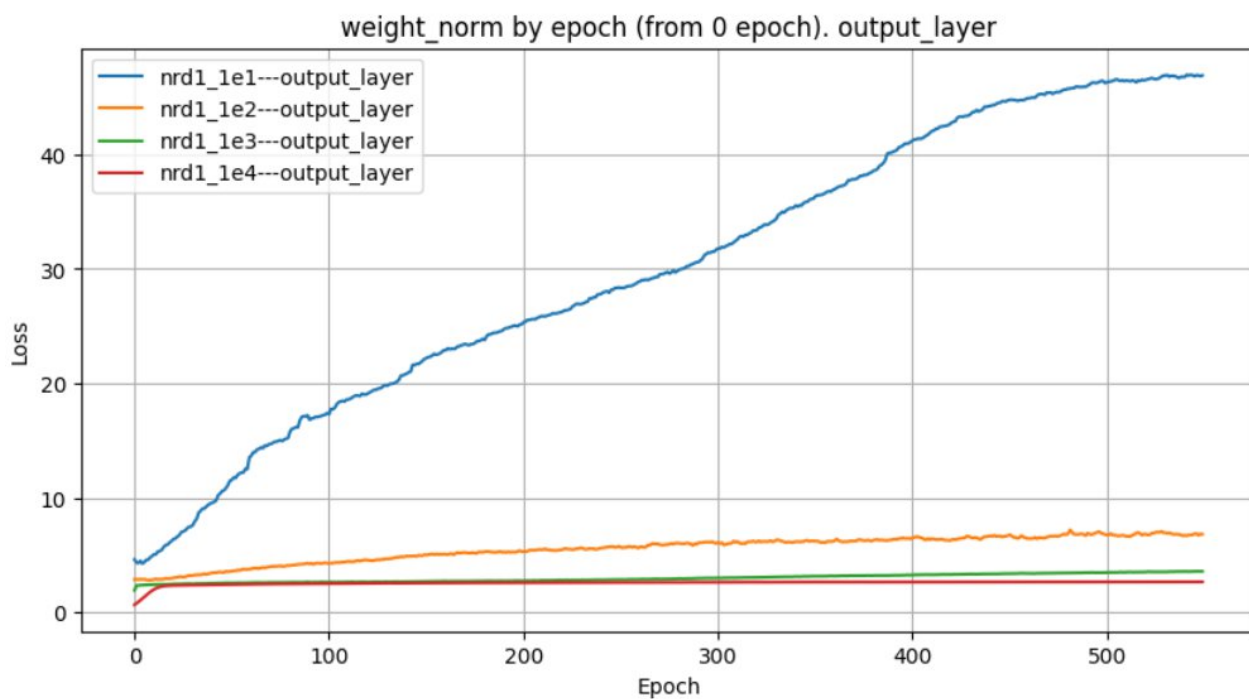
Самым активным среди residual_blocks является слой bn2. На нем текут самые большие градиенты в блоке.



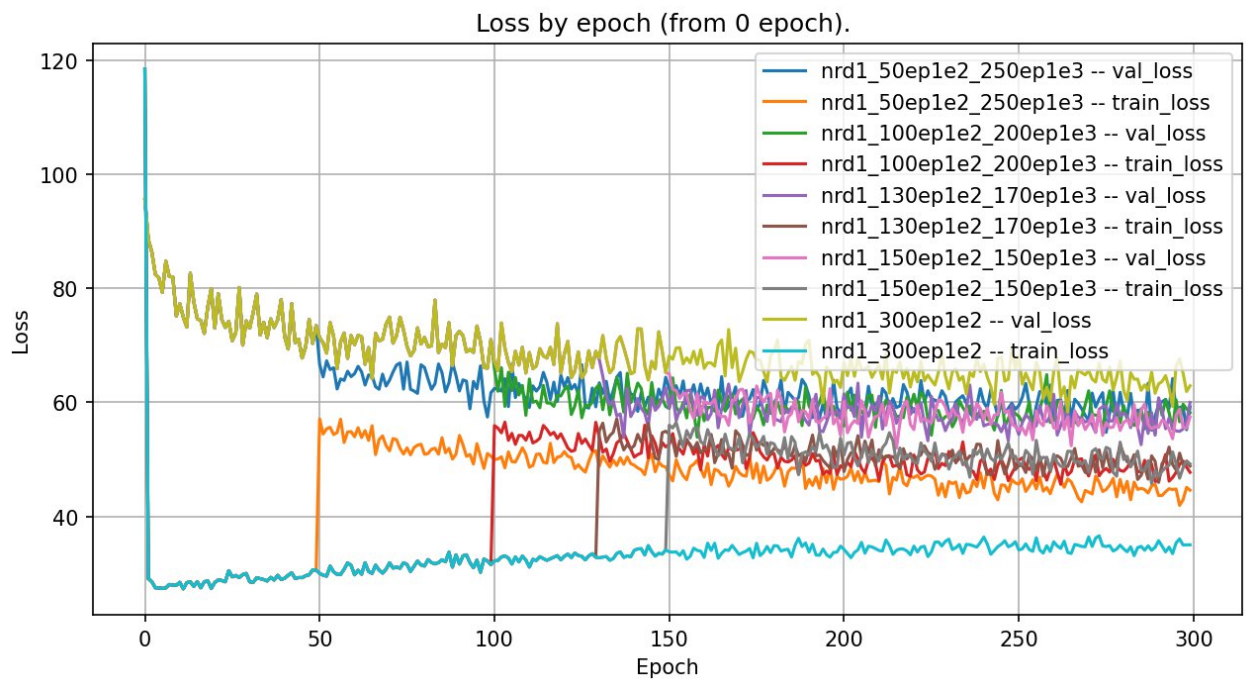


Выходной слой напрямую реагирует на лосс, поэтому на нем всегда большие пляшущие градиенты. Слой активно переписывается и меняется каждую эпоху.

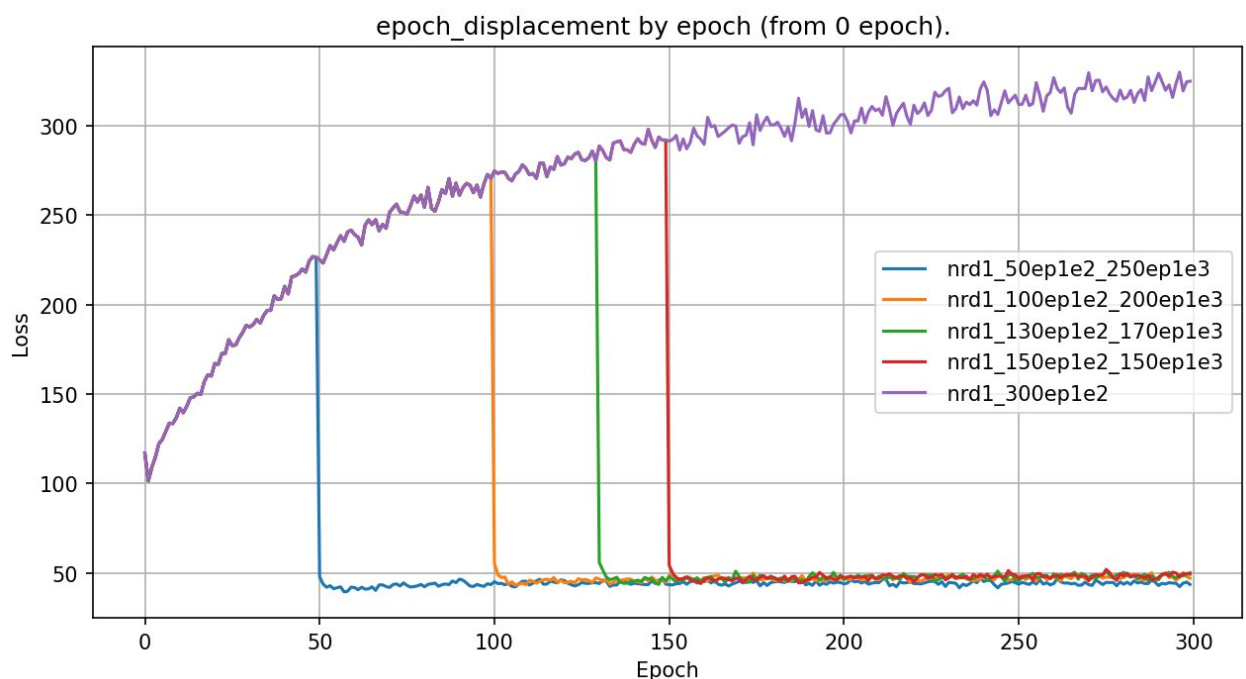


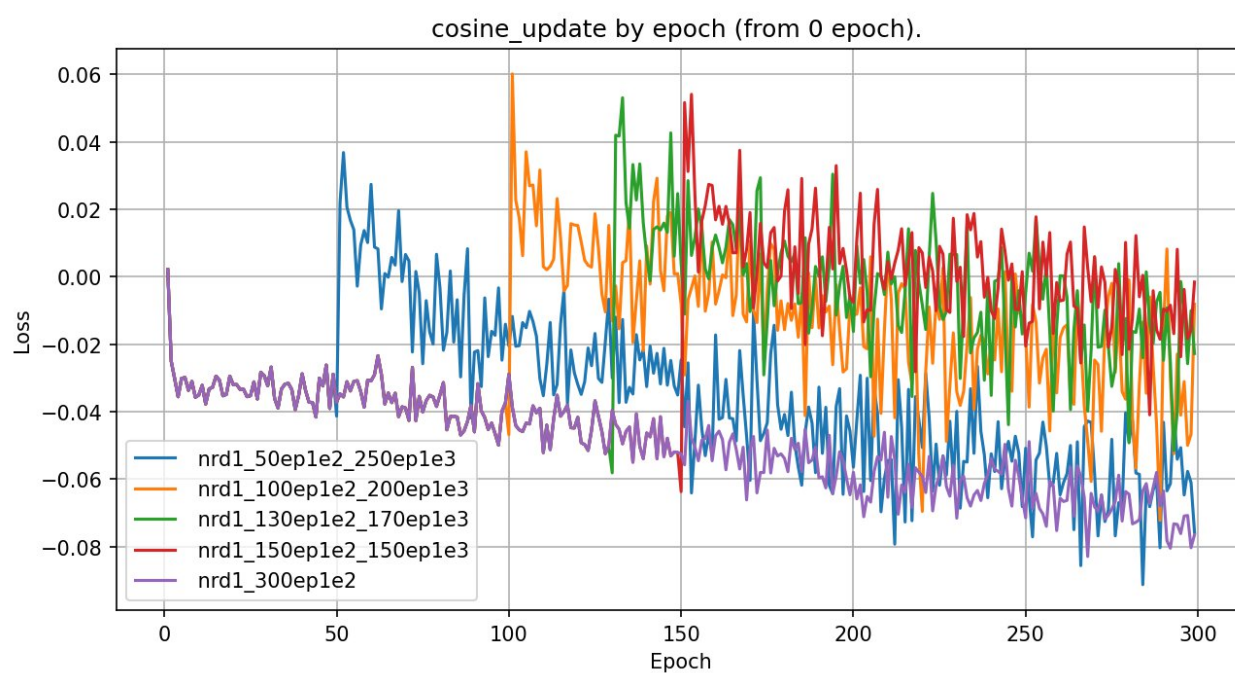
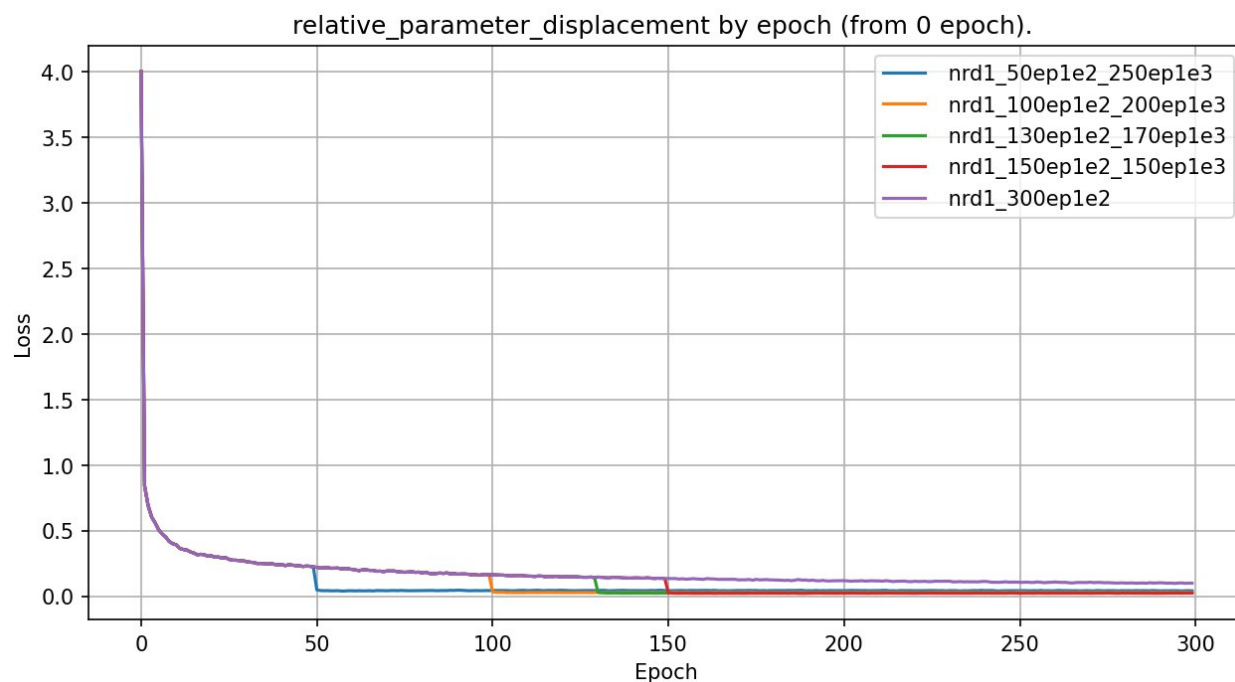


Если добавить переключение LR во время обучения, то кривые обучения очень сильно изменятся

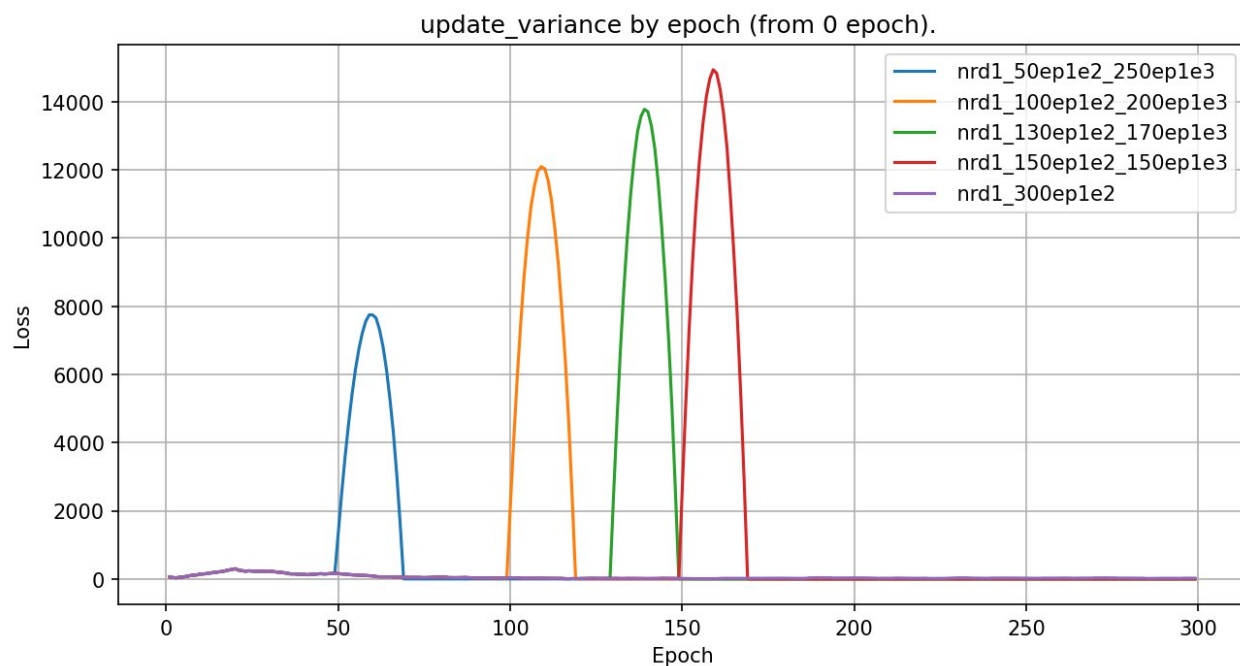


Тут показан результат изменения LR на разных эпохах обучения. Видно, что переключение LR на более низкий приводит к снижению валидационного лосса и увеличению трэйн лосса. Момент переключения имеет значение. Чем он дальше, тем лучше.

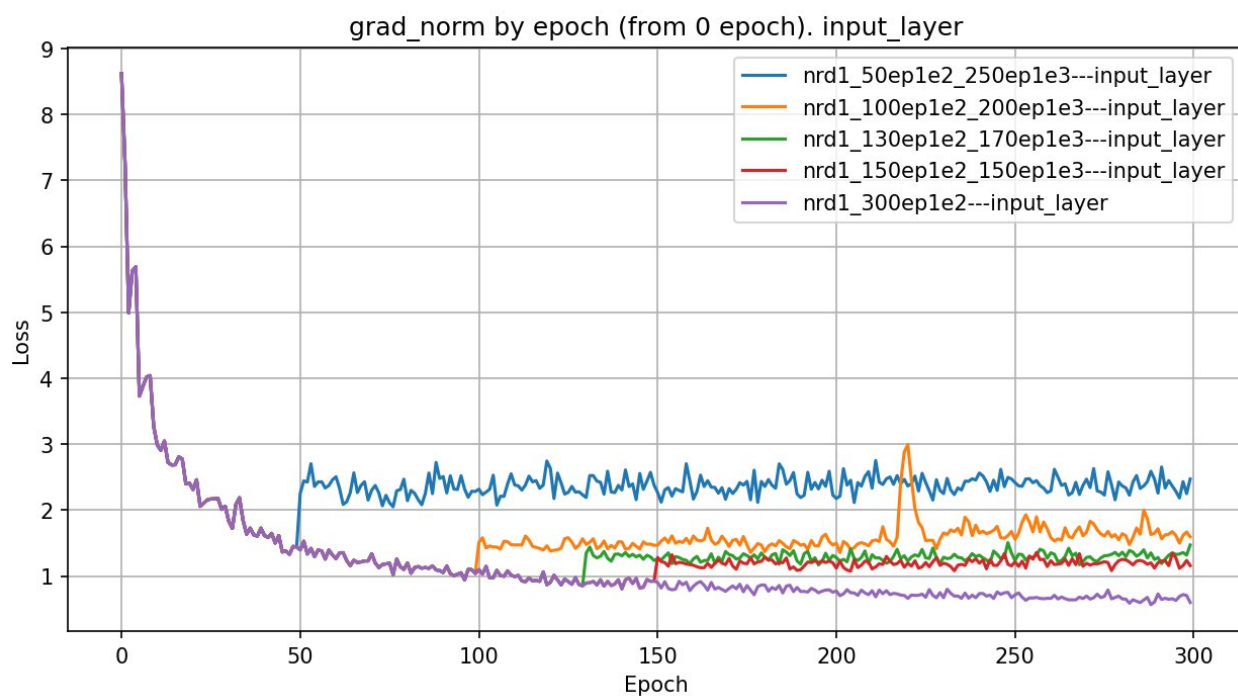


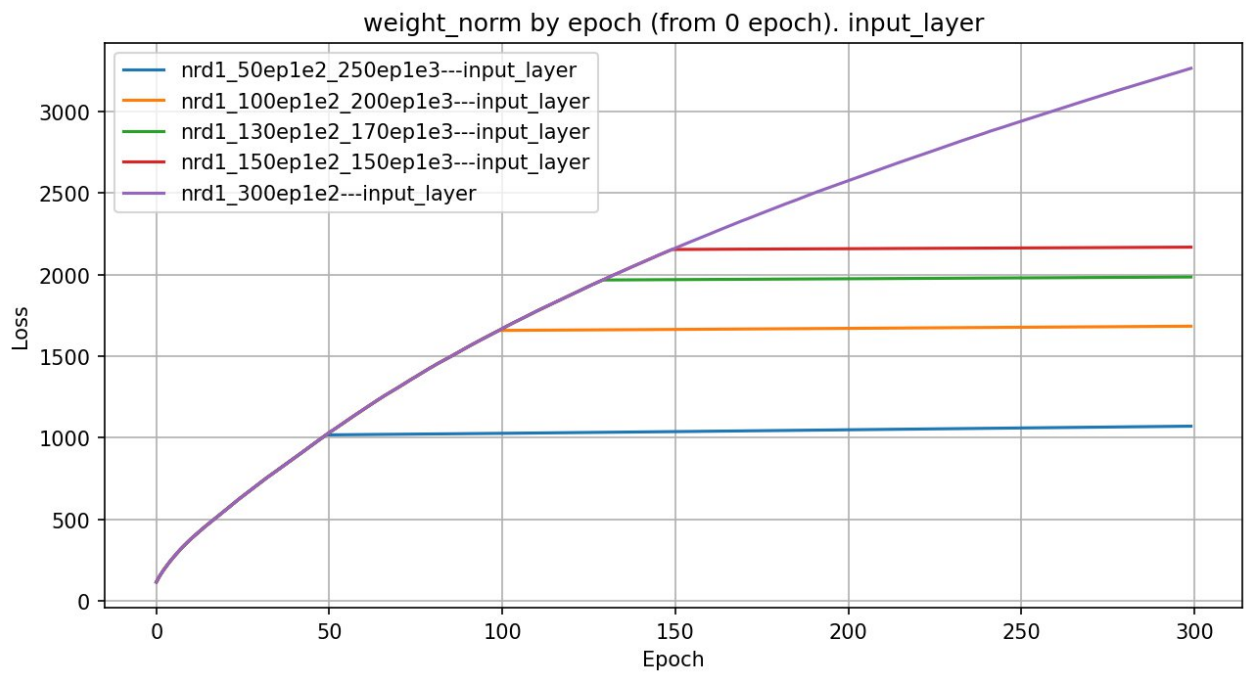


cosine_update - в фазе активного обучения, когда лосс стабильно падает не смотря на шум, cosine_update имеет высокие значения. Но потом, т.к. каждая эпоха валидируется на новом наборе данных - обновления сильно зашумлены - cosine_update падает. Видно, что в целом при низкой скорости обучения cosine_update положительный - улучшение лосса идет более целенаправленно.

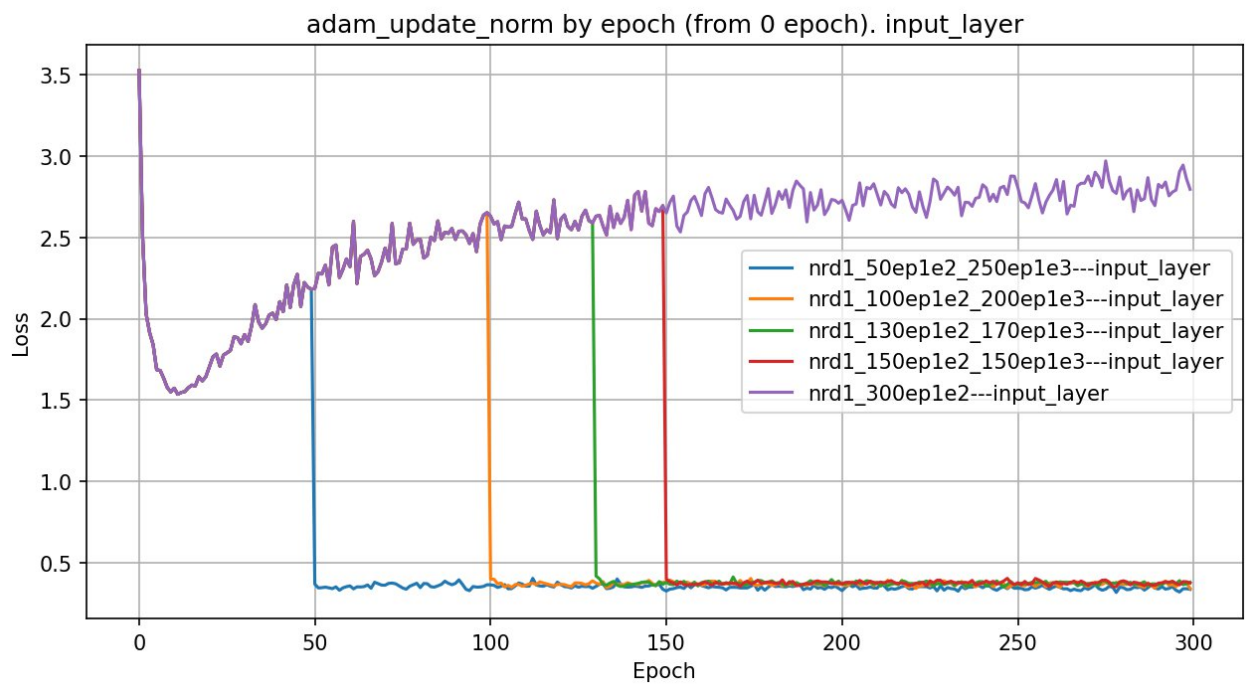


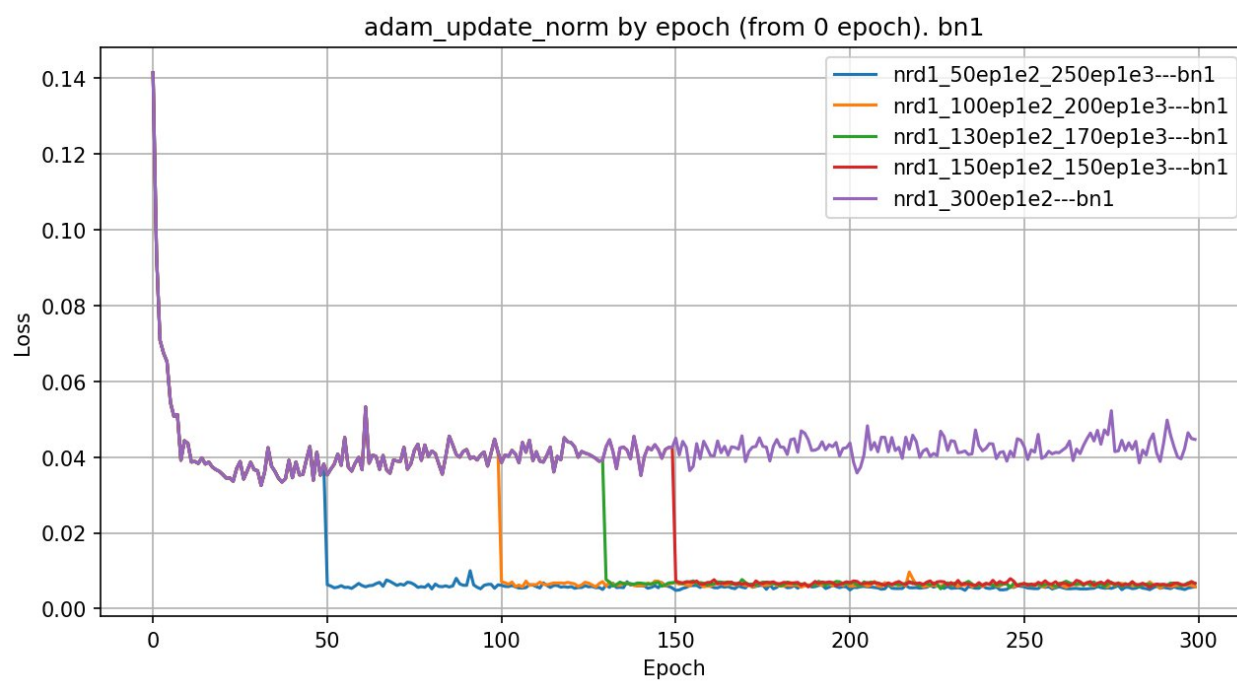
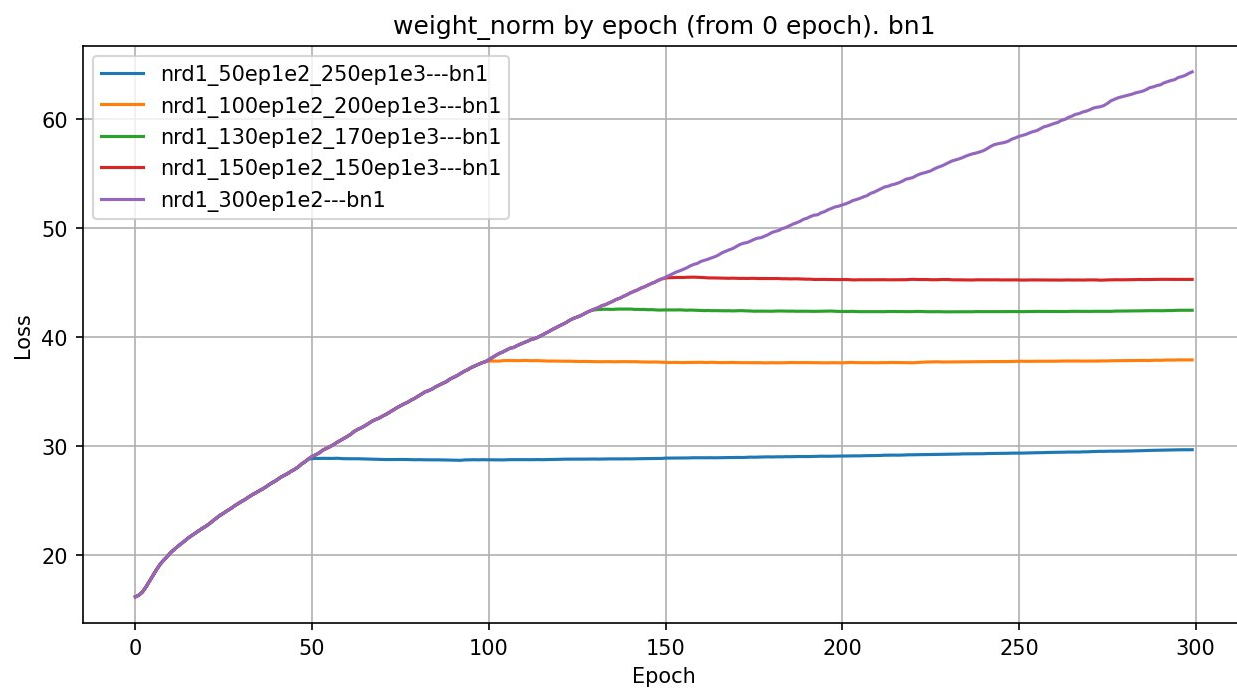
Переключение LR приводит к резкому снижению изменения параметров модели (весов).

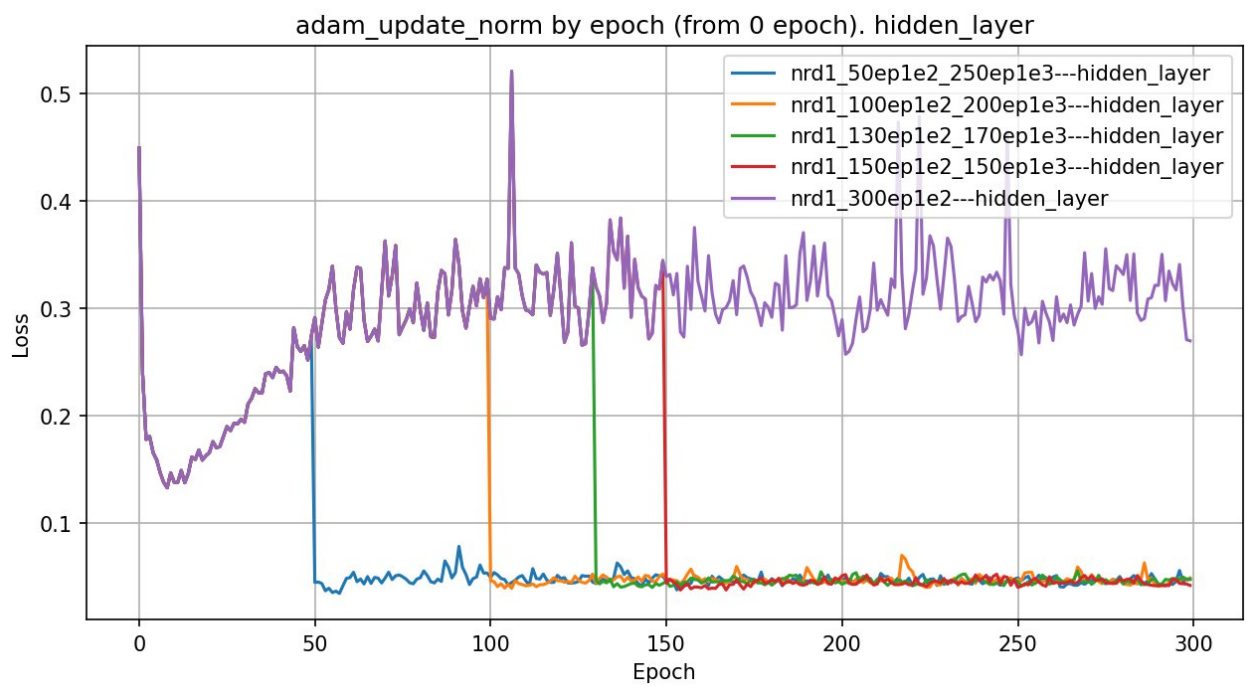
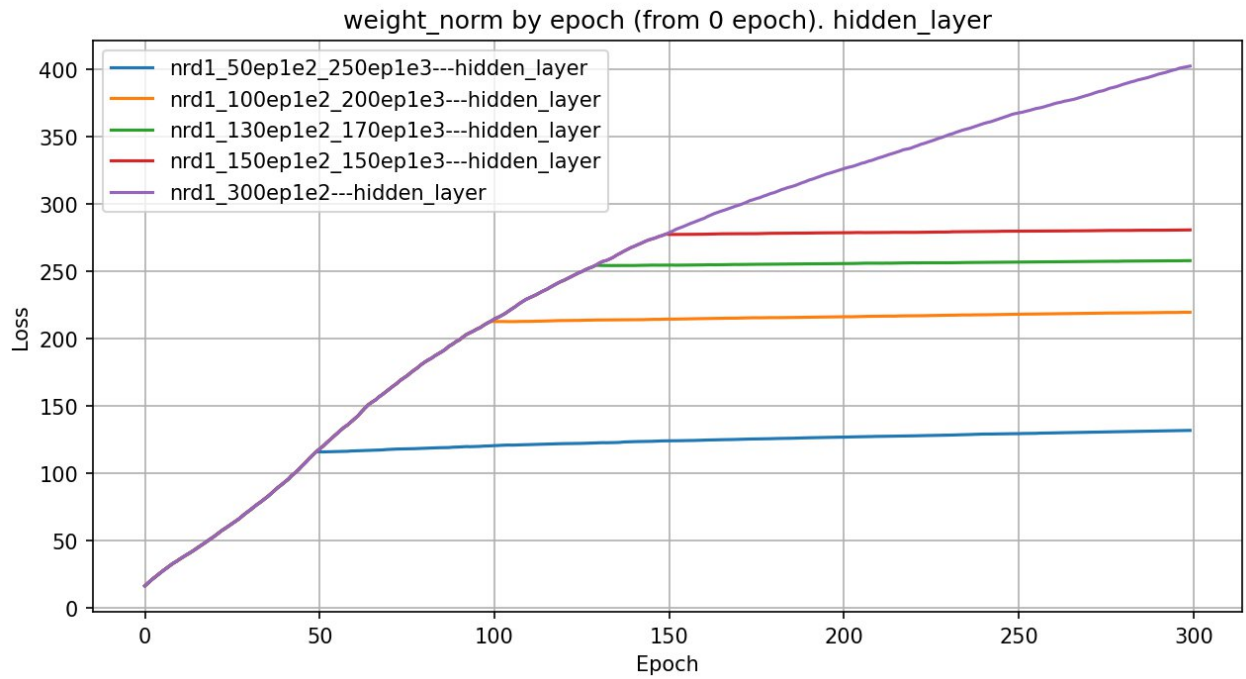


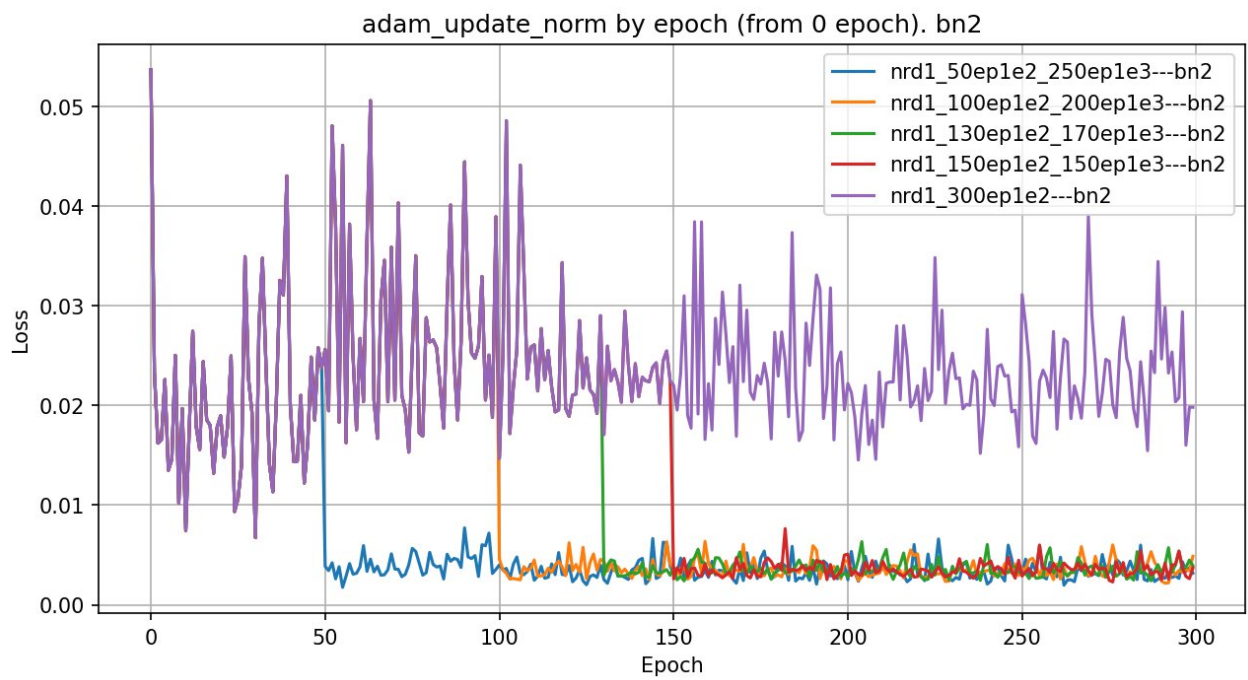
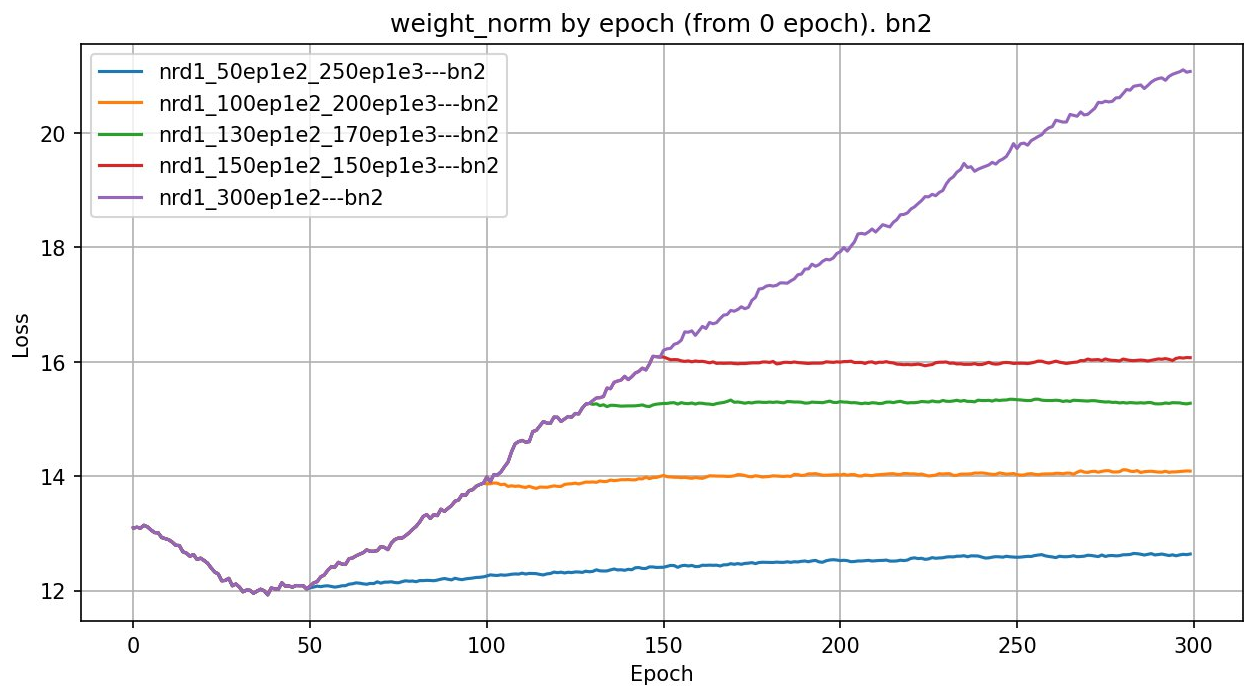


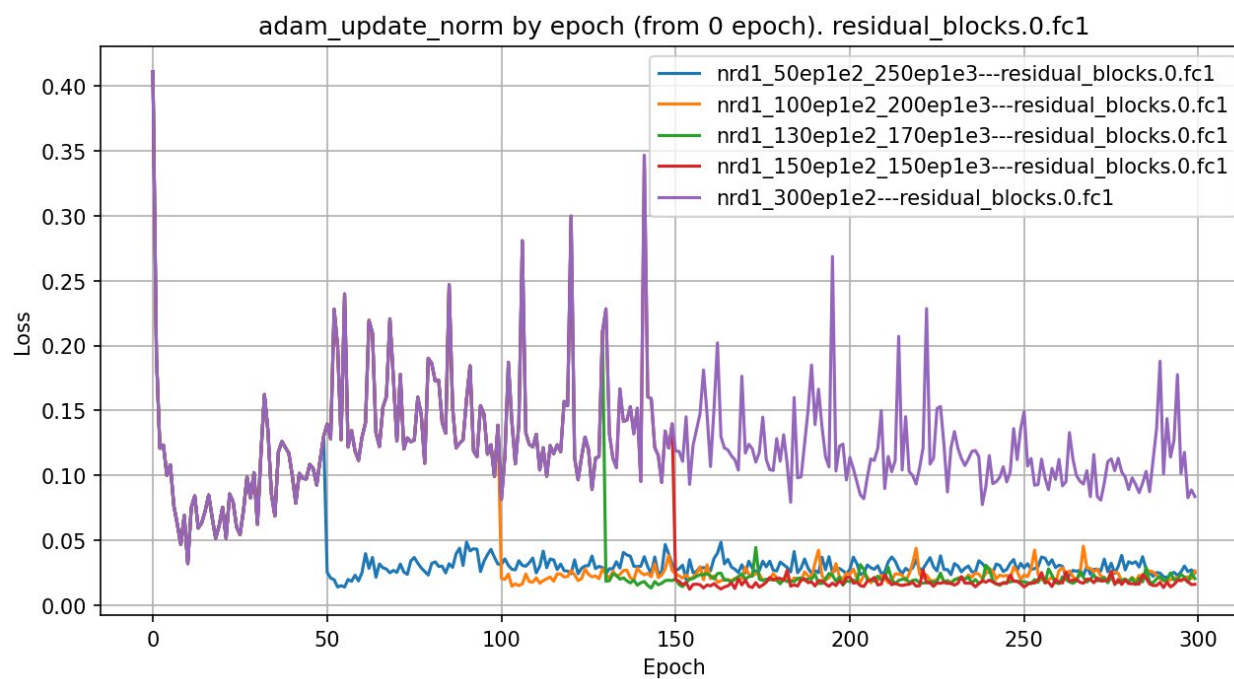
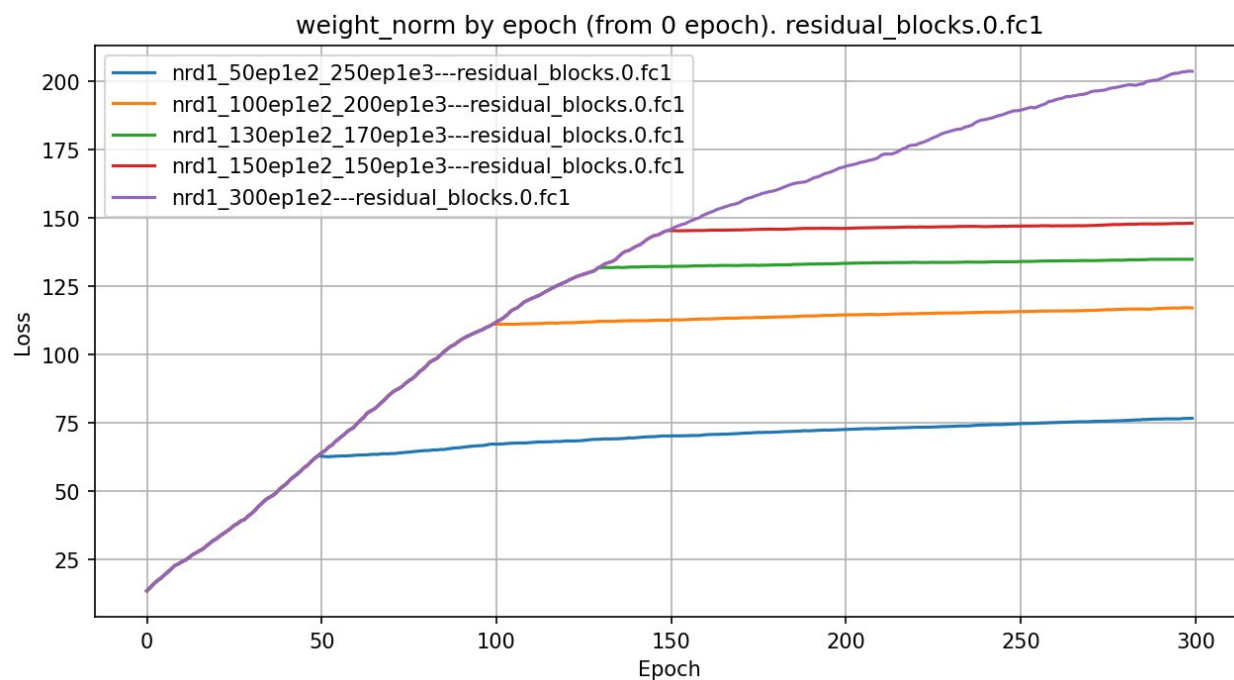
Видно, что пре переключении LR входной слой выходит из инфляционного режима.
Другие слои тоже.

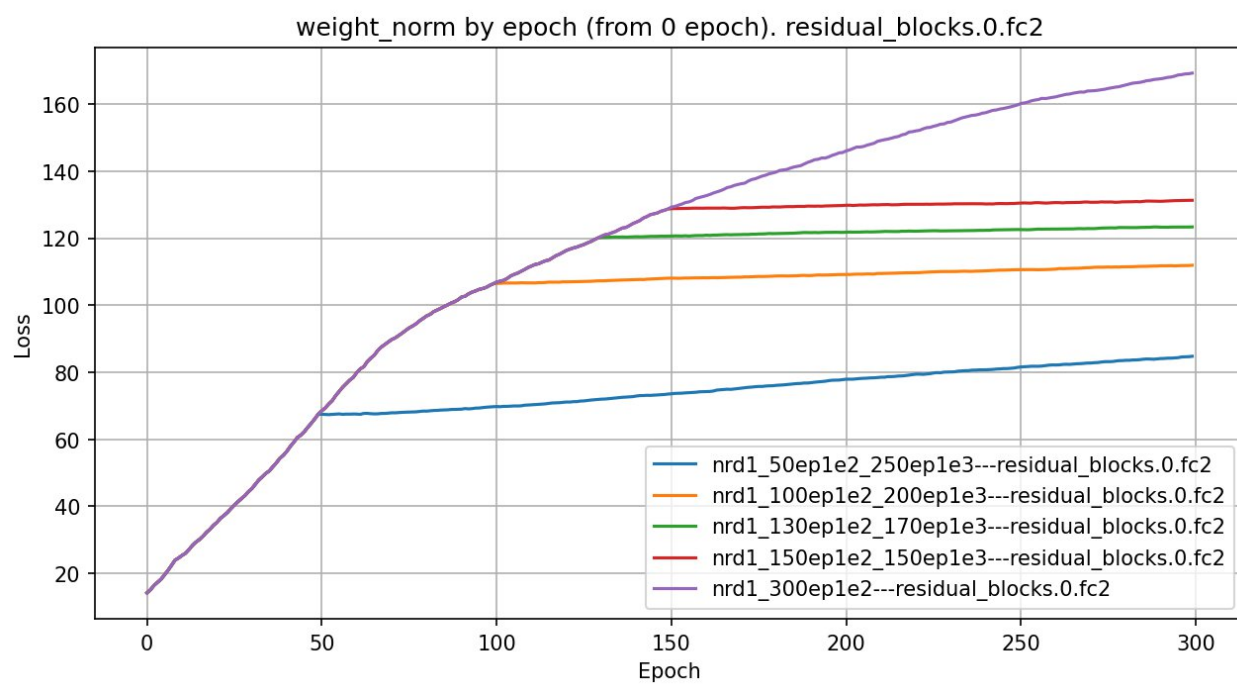
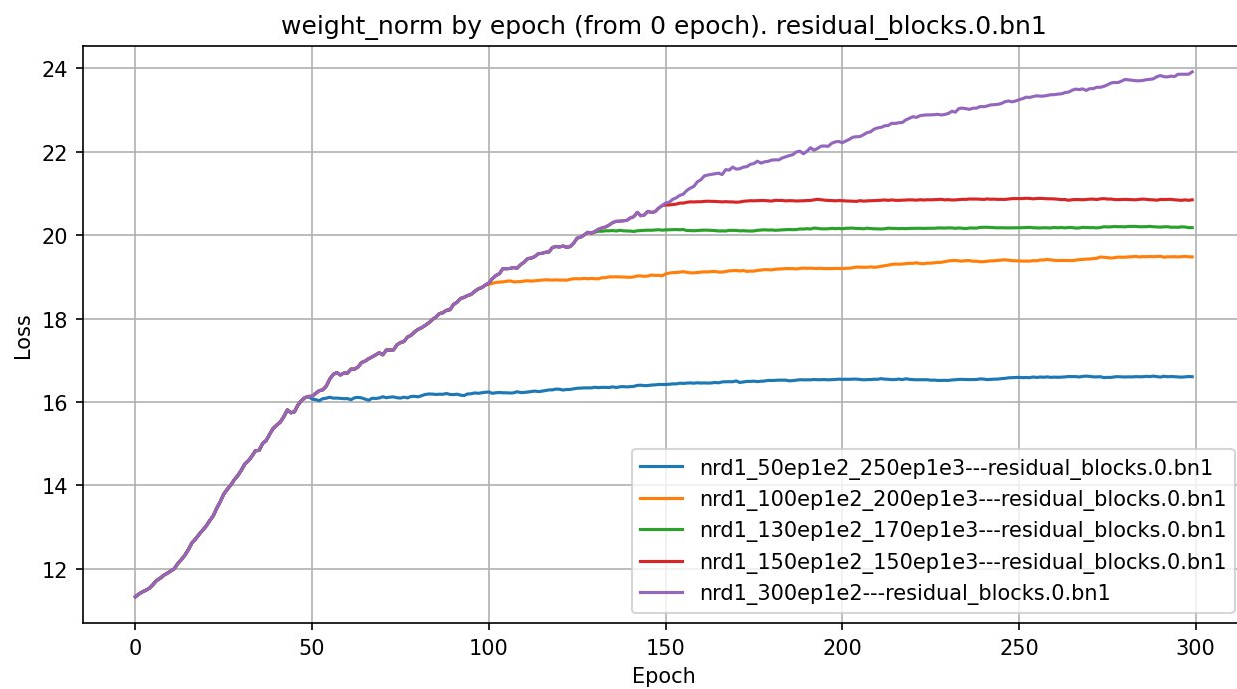


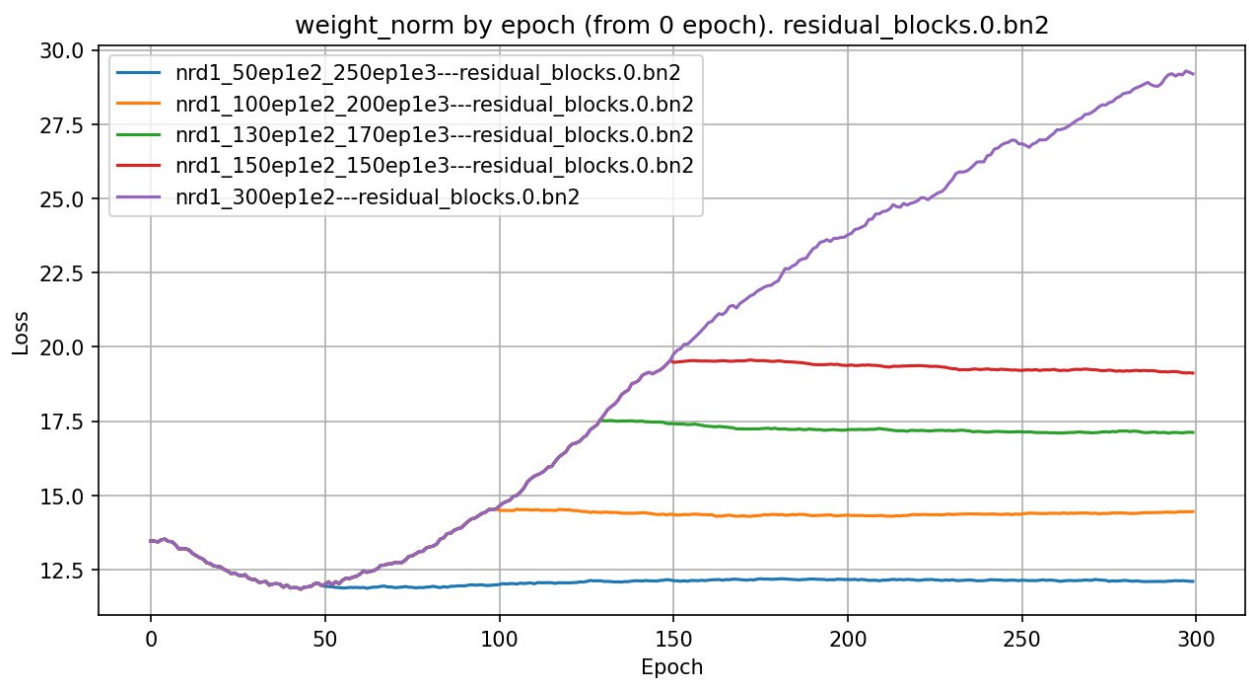
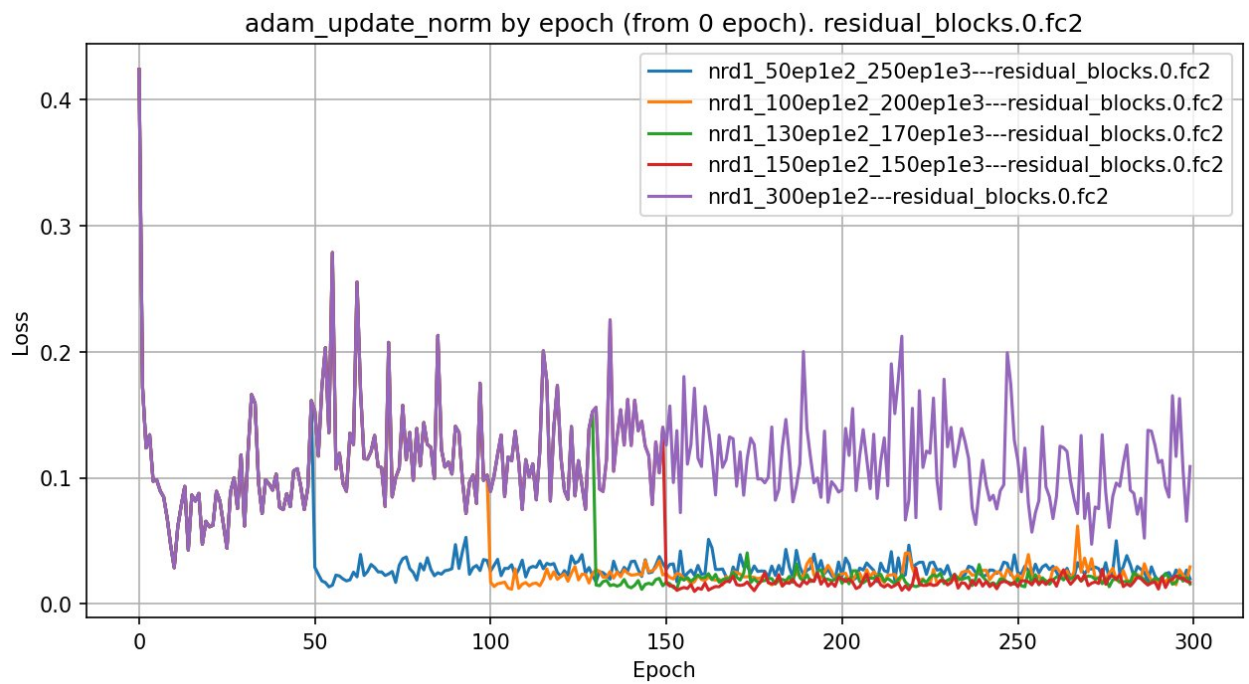


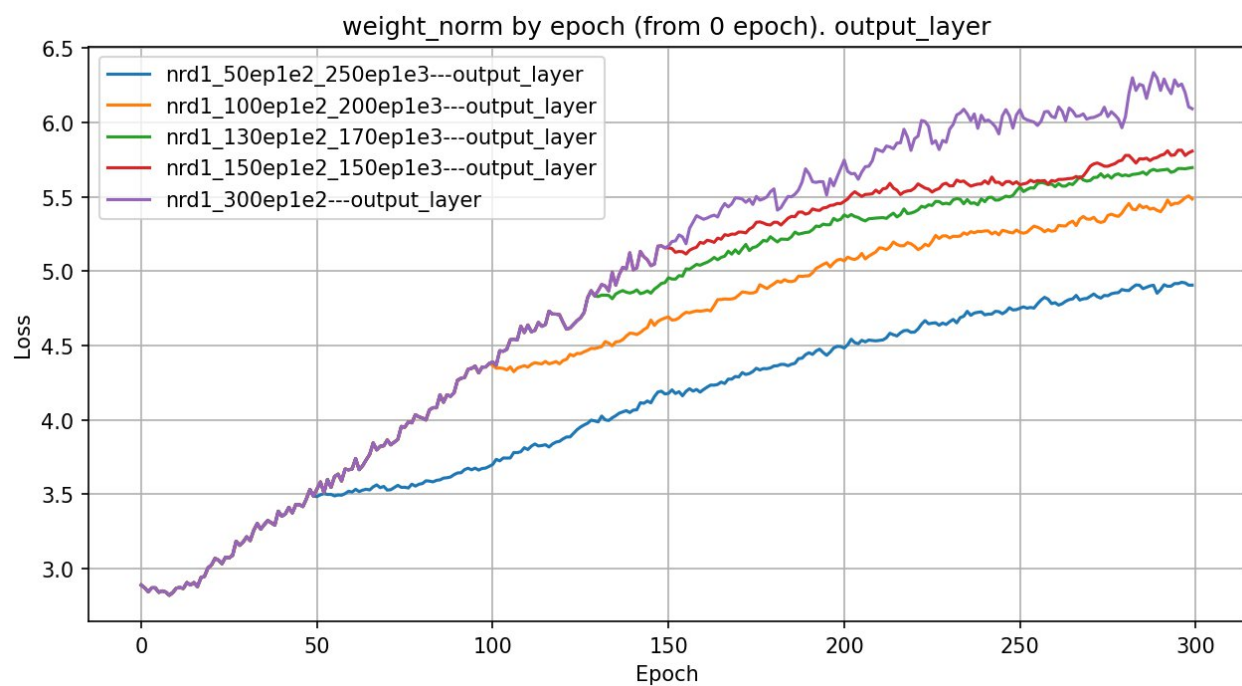
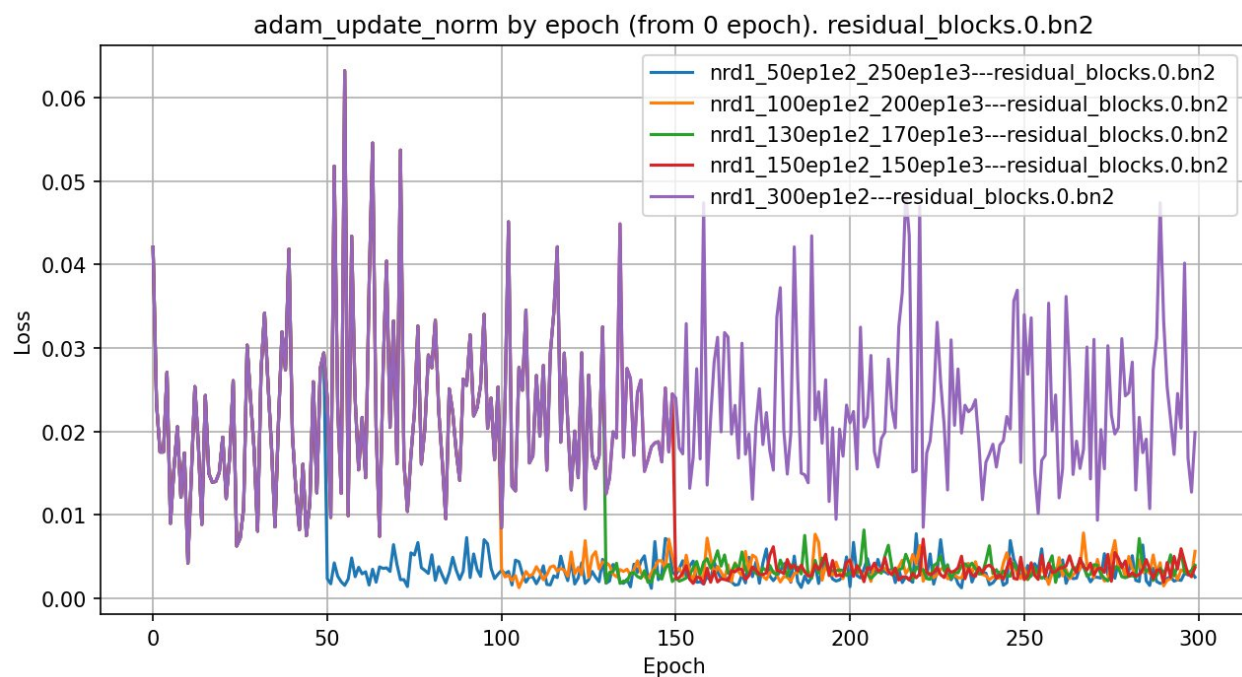


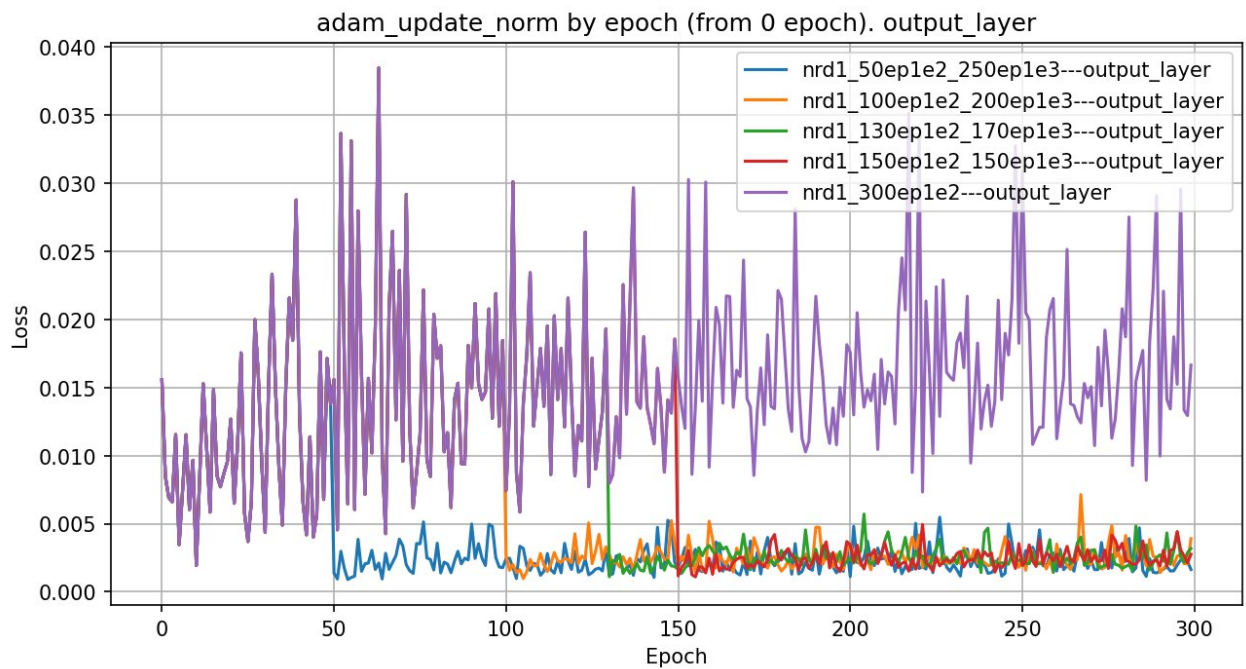










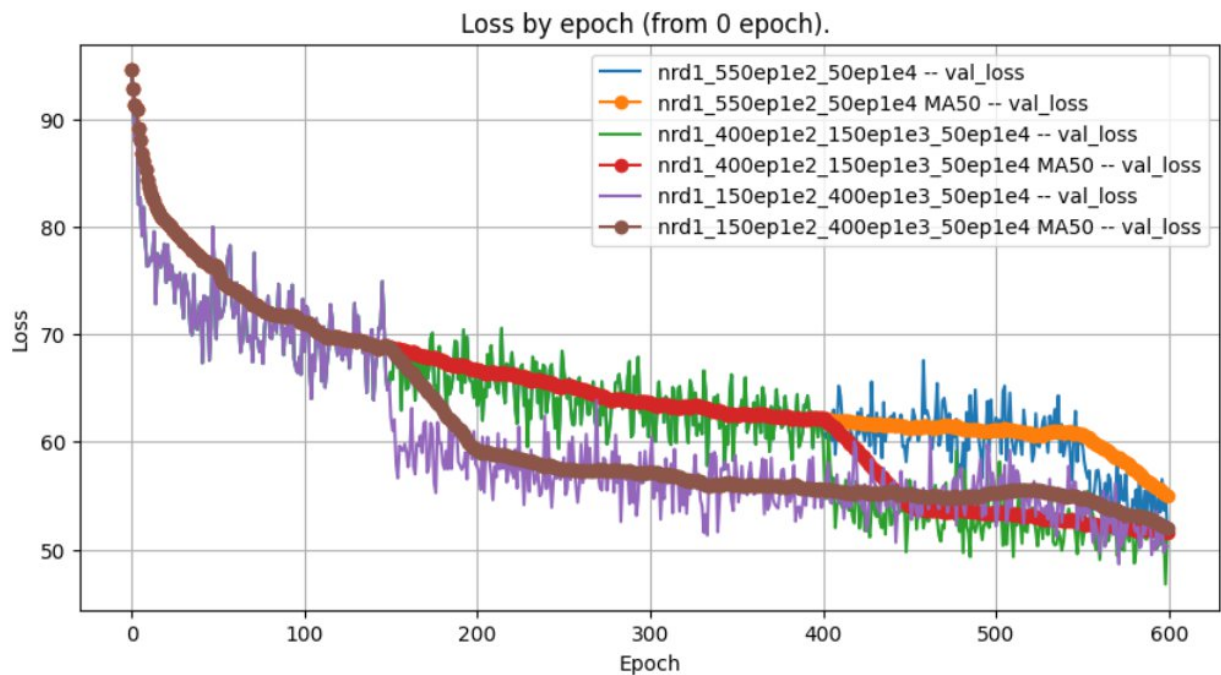


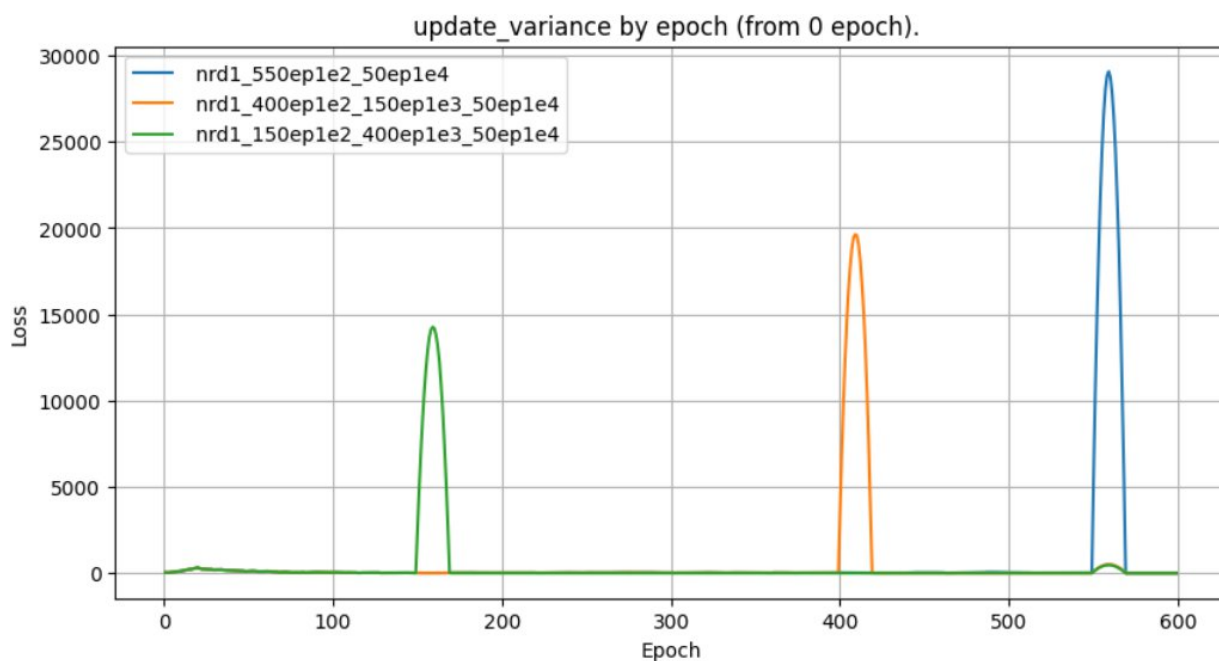
Видно, что батчнорм слои bn2 и output_layer имеют «ямку» в кривой изменения весов.

Ямка наблюдается на ранних эпохах.

Для линейных слоев характерна «ямка» для adam_update_norm, что в сочетании со стремительно растущими весами и слабо меняющемся лоссе указывает на «инфляцию весов». Т.е. при высоких LR линейные слои быстро учатся в первые эпохи, а потом уходят в режим инфляции.

Что будет, если сдвиг отложить?





Beam search и лучшая эпоха

В таблице ниже приведены запуски со стартом $1e-2$, переключением на $1e-3$ на 150 и 400 эпохе, переключением на $1e-4$ на последних 50 эпохах (чтобы улучшить лосс и результат для beam search)

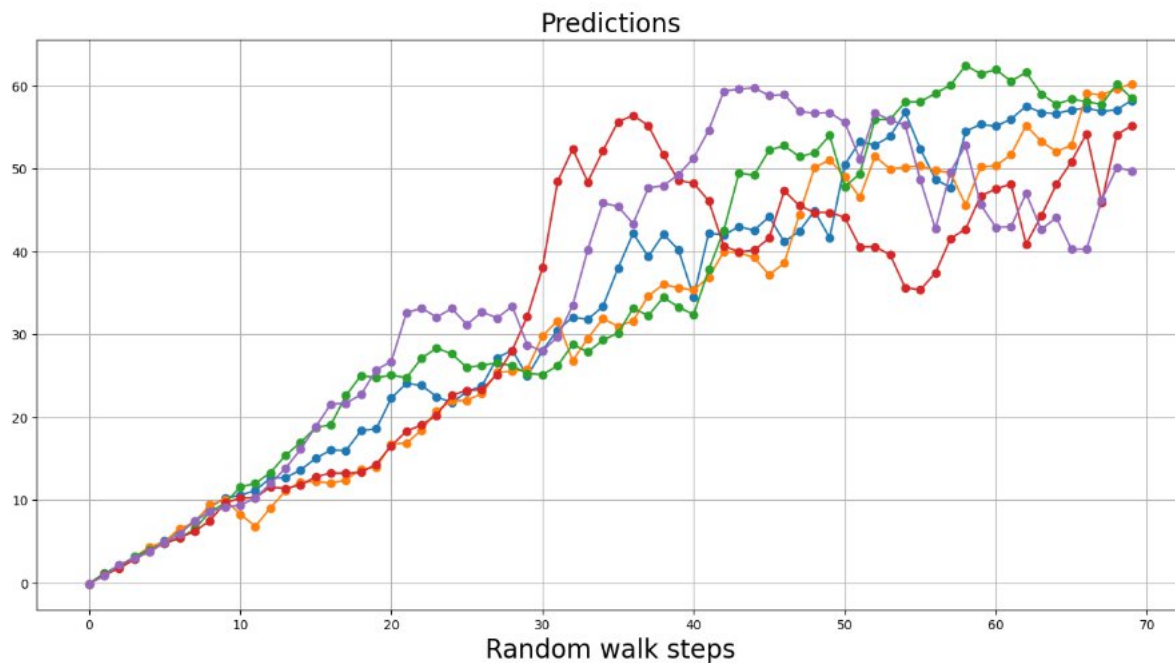
run	lr_schedule	num_epochs	epoch	is_best	MA50	solv%	mean	std
254	0: $1e-2$, 400: $1e-3$, 550: $1e-4$	600	599	0	51.51	100	311.86	100.80
244	0: $1e-2$, 400: $1e-3$, 550: $1e-4$	600	598	1	51.53	100	308.45	76.43
254	0: $1e-2$, 400: $1e-3$, 550: $1e-4$	600	549	0	52.54	99	314.25	58.22
248	0: $1e-2$, 550: $1e-4$	600	599	0	53.26	98	339.03	92.9
249	0: $1e-2$, 150: $1e-3$, 550: $1e-4$	600	599	0	55.03	100	280.64	61.17
251	0: $1e-2$, 150: $1e-3$, 550: $1e-4$	600	571	1	52.02	100	290.71	72.75
251	0: $1e-2$, 150: $1e-3$, 550: $1e-4$	600	549	0	53.54	99	335.5	80.91
251	0: $1e-2$, 150: $1e-3$, 550: $1e-4$	600	442	1	54.76	97	357.5	99.92
261	0: $1e-3$, 550: $1e-4$	600	599	0	55.22	94	378.79	123.13

Видно, что избыточная тренировка с высоким LR уменьшает лосс, но результат по beam search хуже.

Лучшая эпоха дает выигрыш, если она рядом с нужной (пороговой эпохой).

Разница между лучшей и не лучшей эпохами хорошо заметна в предсказаниях на тестовом наборе.

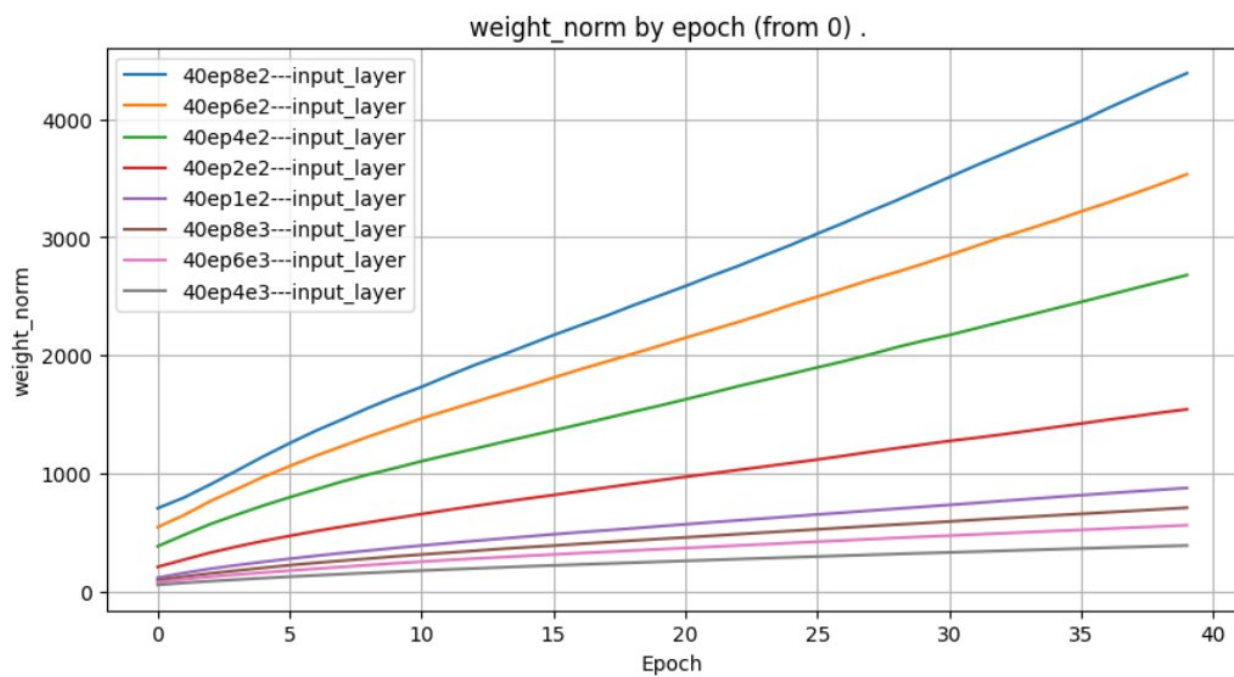
Эпоха 549 (запуск 252, кривая 0: $1e-2$, 150: $1e-3$, 550: $1e-4$)

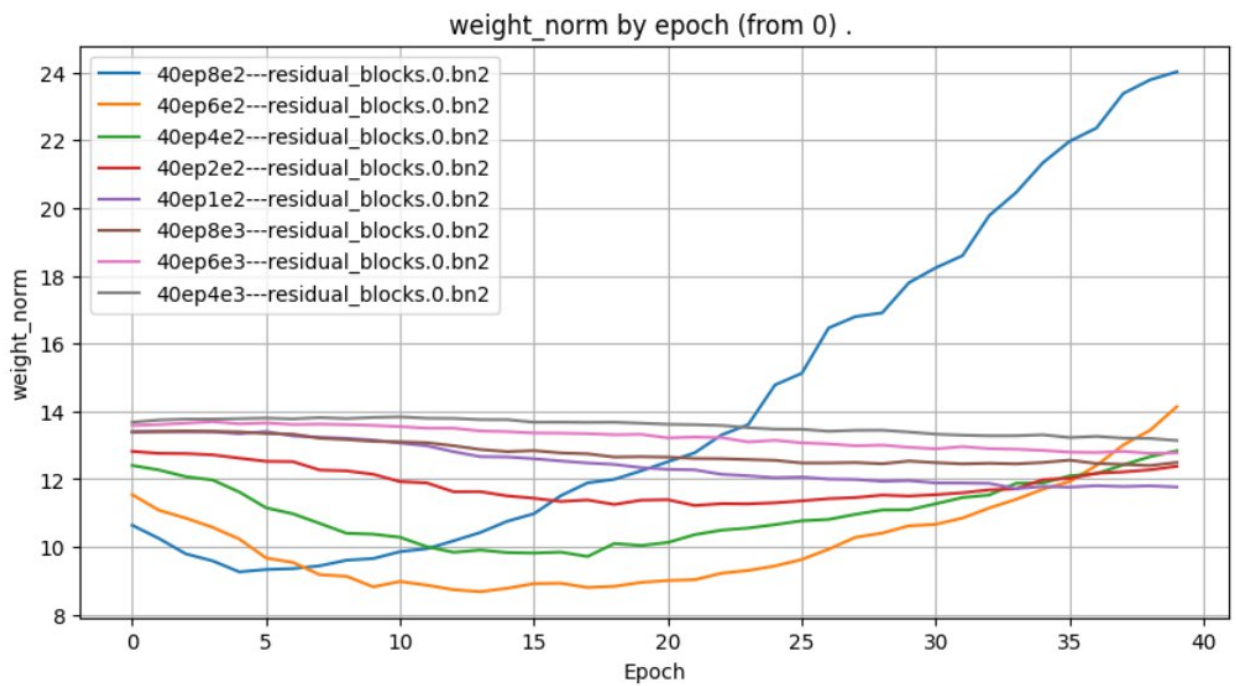
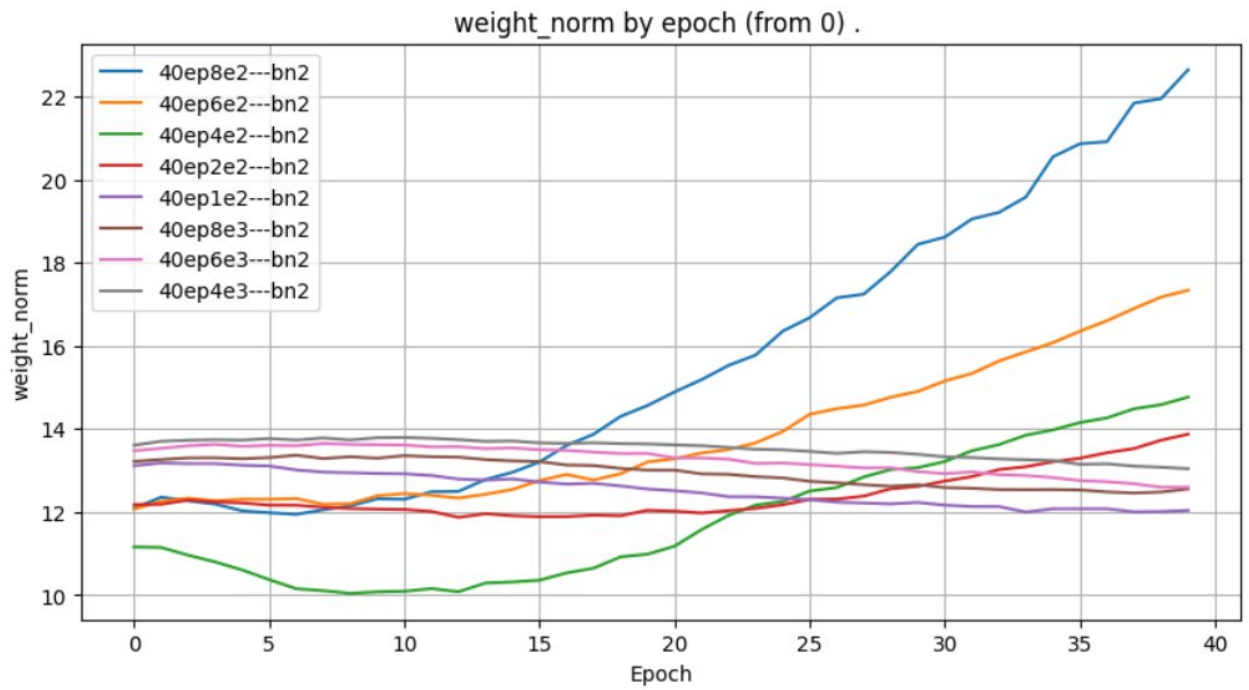


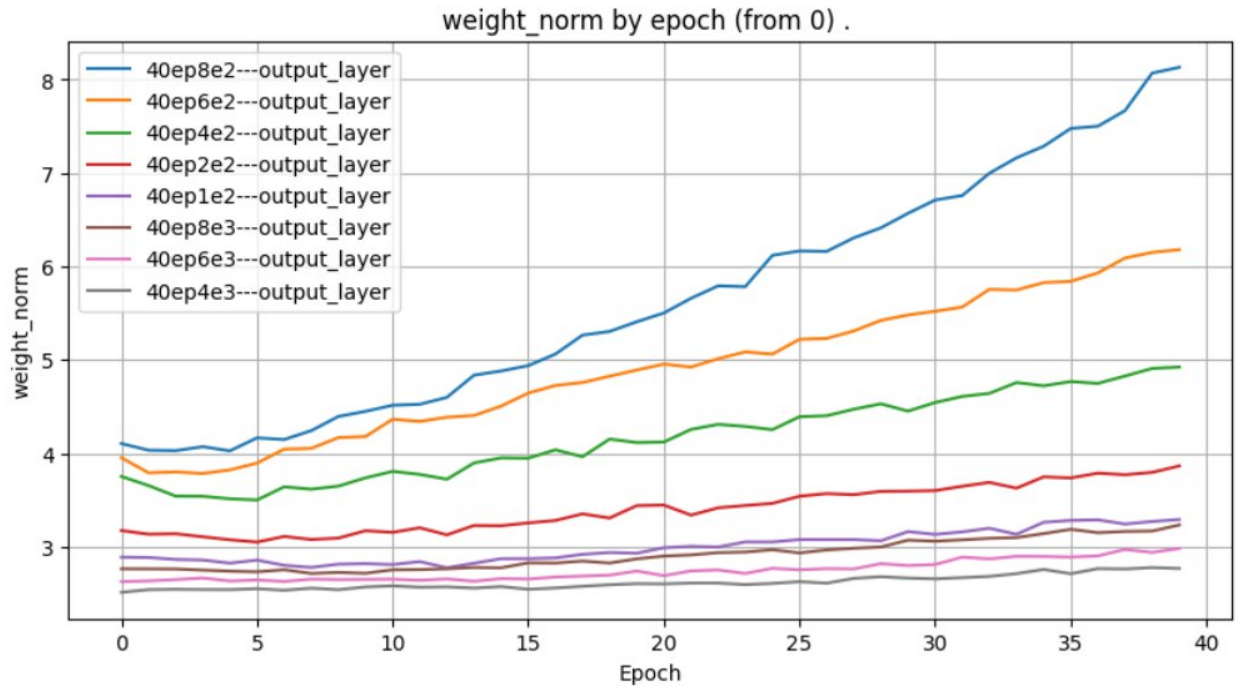
Лучшая эпоха 442 (запуск 251, кривая 0: $1e-2$, 150: $1e-3$, 550: $1e-4$)



Просмотр первых 40 эпох с метриками





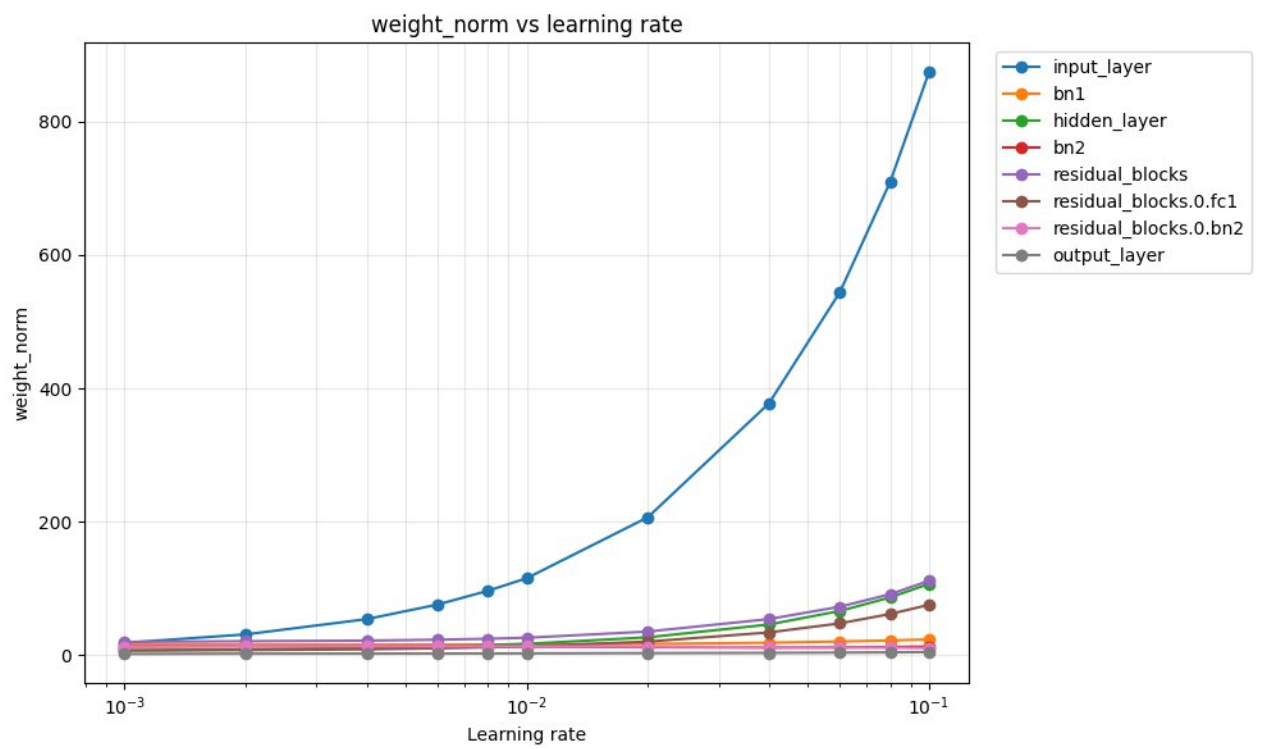
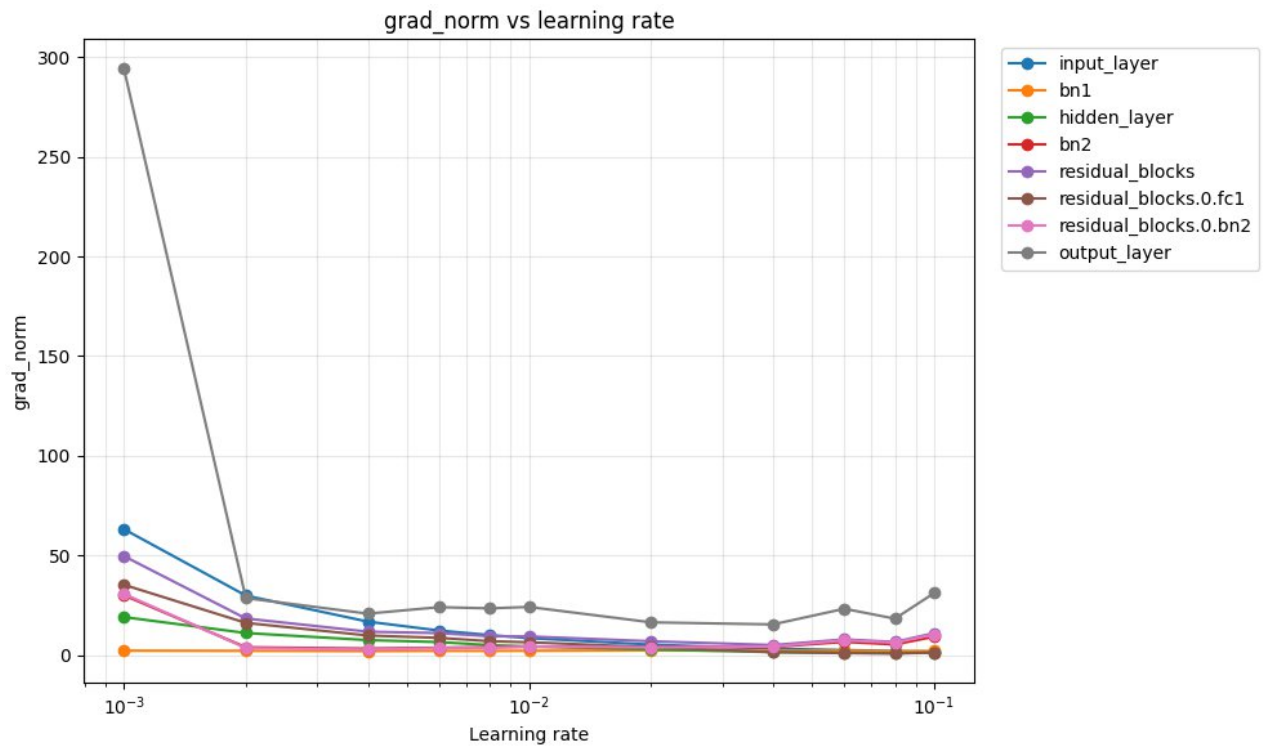


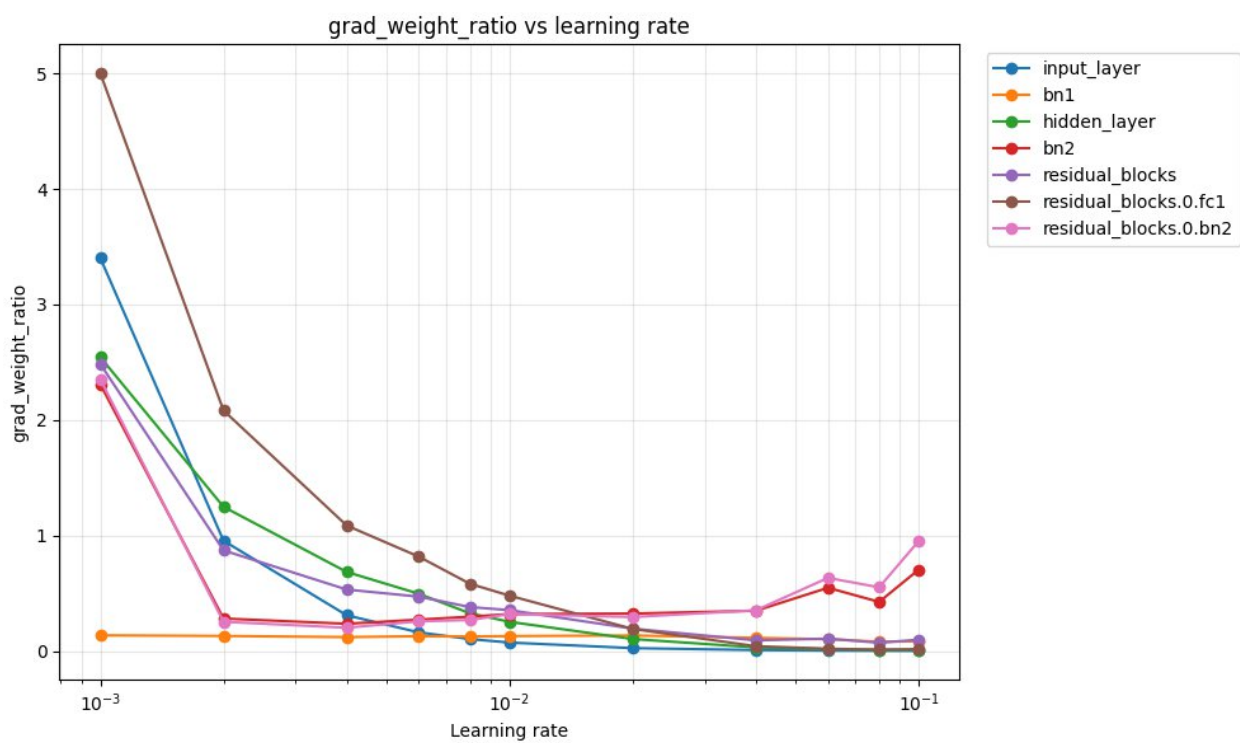
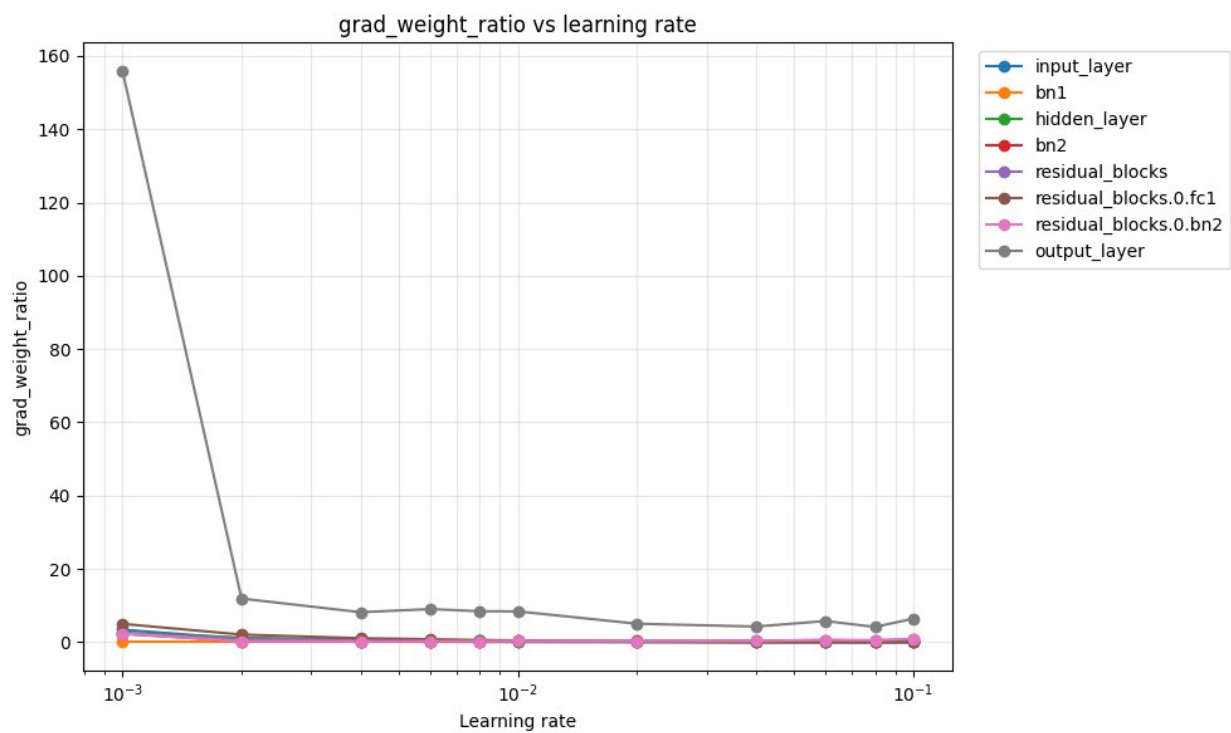
Судя по этим данным, модель при LR $1e-4$ и даже $1e-6$ учится чему-то полезному, т.к. мы наблюдаем «ямку» в весах на батчнорм слоях.

Интересным экспериментом получилось усреднение результатов 20 запусков при разных LR. Видно, что есть некоторый интервал оптимальности, в котором разумно начинать тренировку.

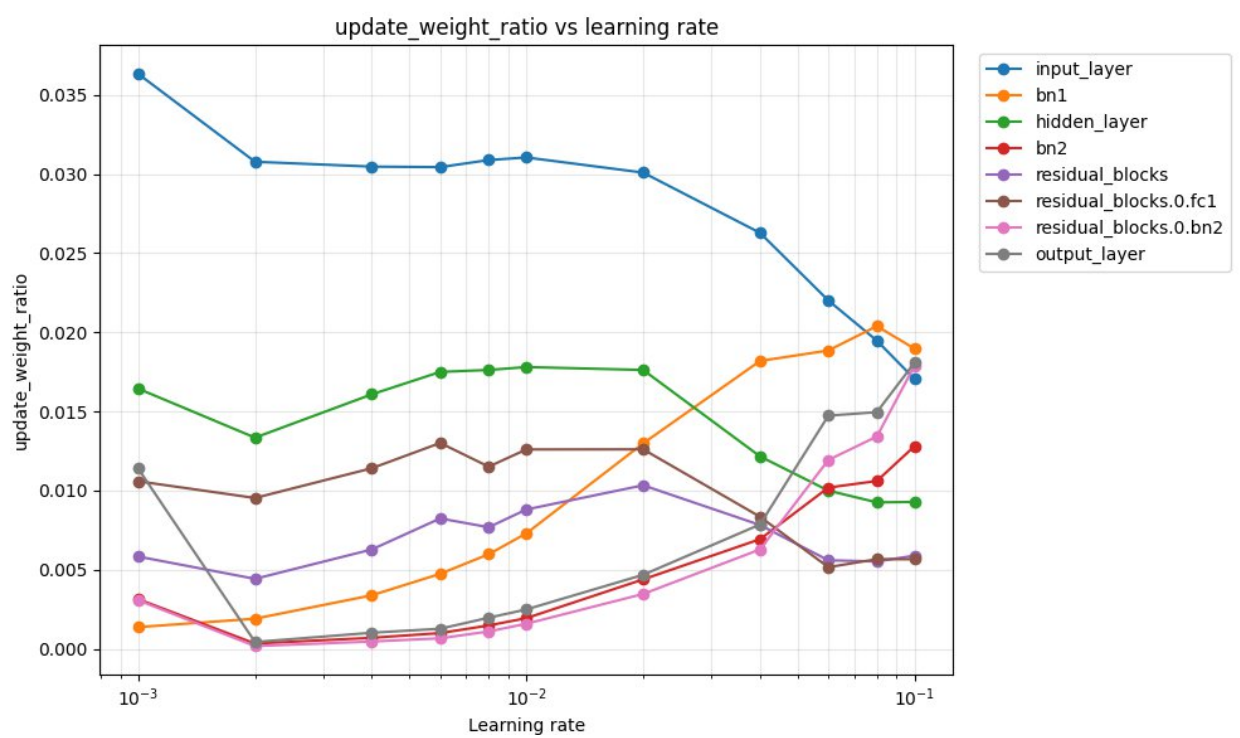
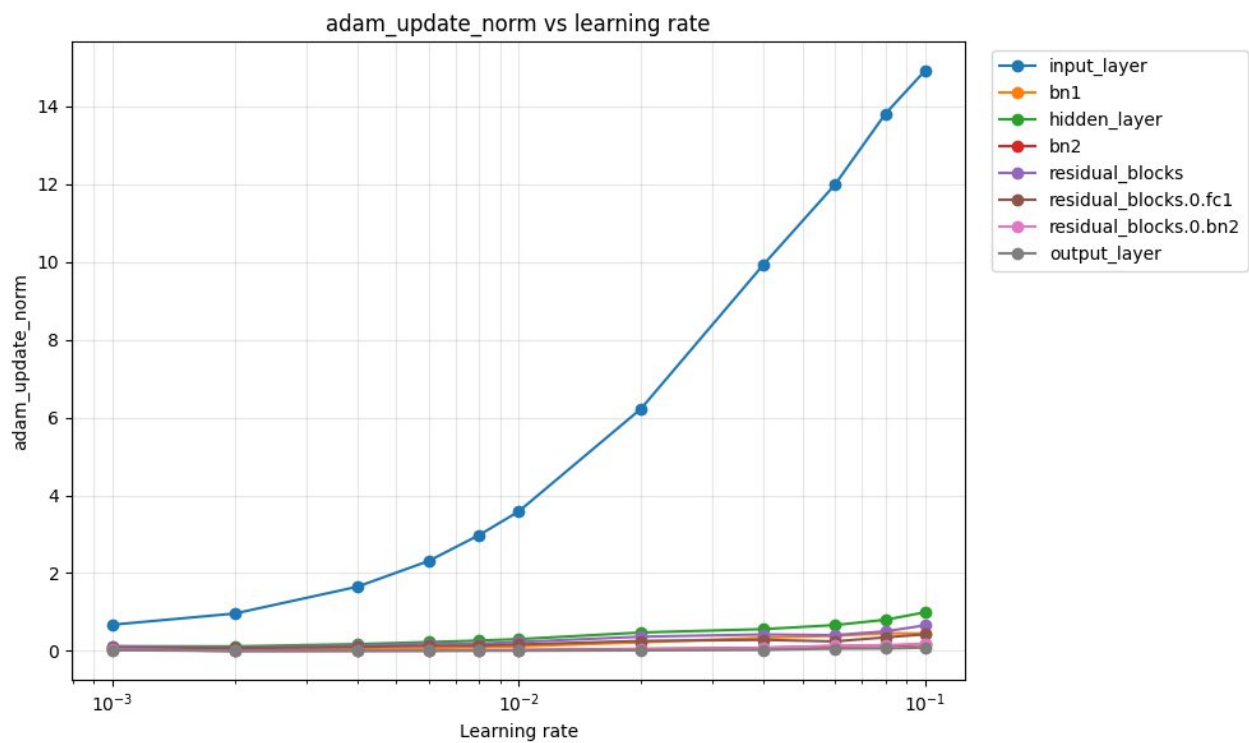
NRD=1

	lr	avg_val_loss	avg_train_loss
0	0.100	98.902923	80.631468
1	0.080	96.008828	81.390111
2	0.060	95.625547	82.022146
3	0.040	94.014473	83.743361
4	0.020	93.445573	94.433324
5	0.010	92.840406	124.015447
6	0.008	92.032383	141.430123
7	0.006	93.265795	171.560957
8	0.004	92.404260	224.173713
9	0.002	93.513739	403.223135
10	0.001	228.439033	709.930707



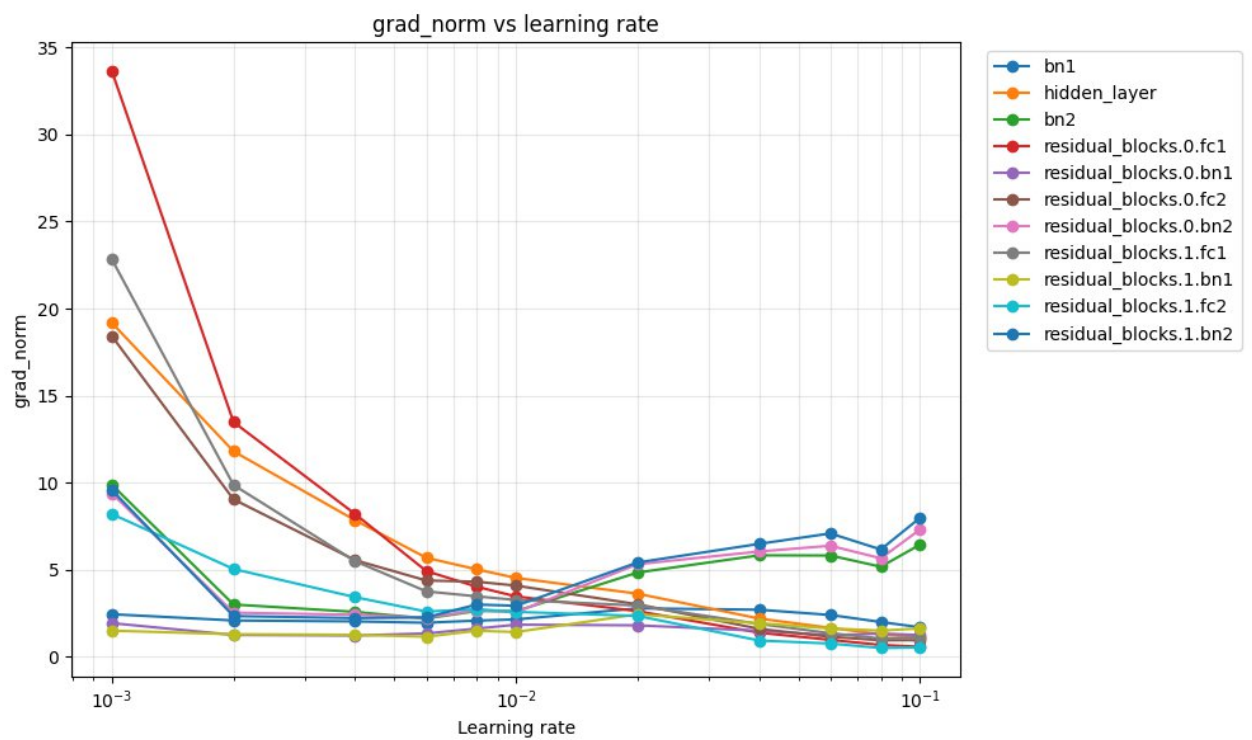
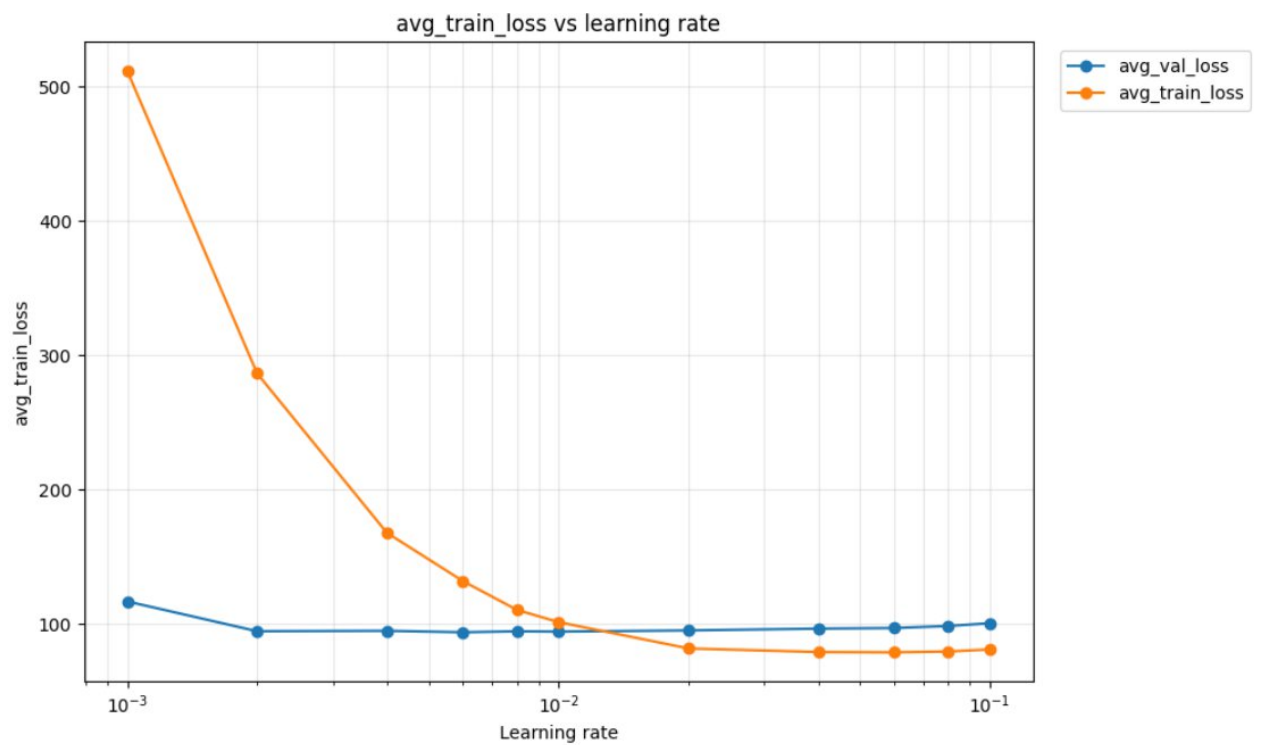


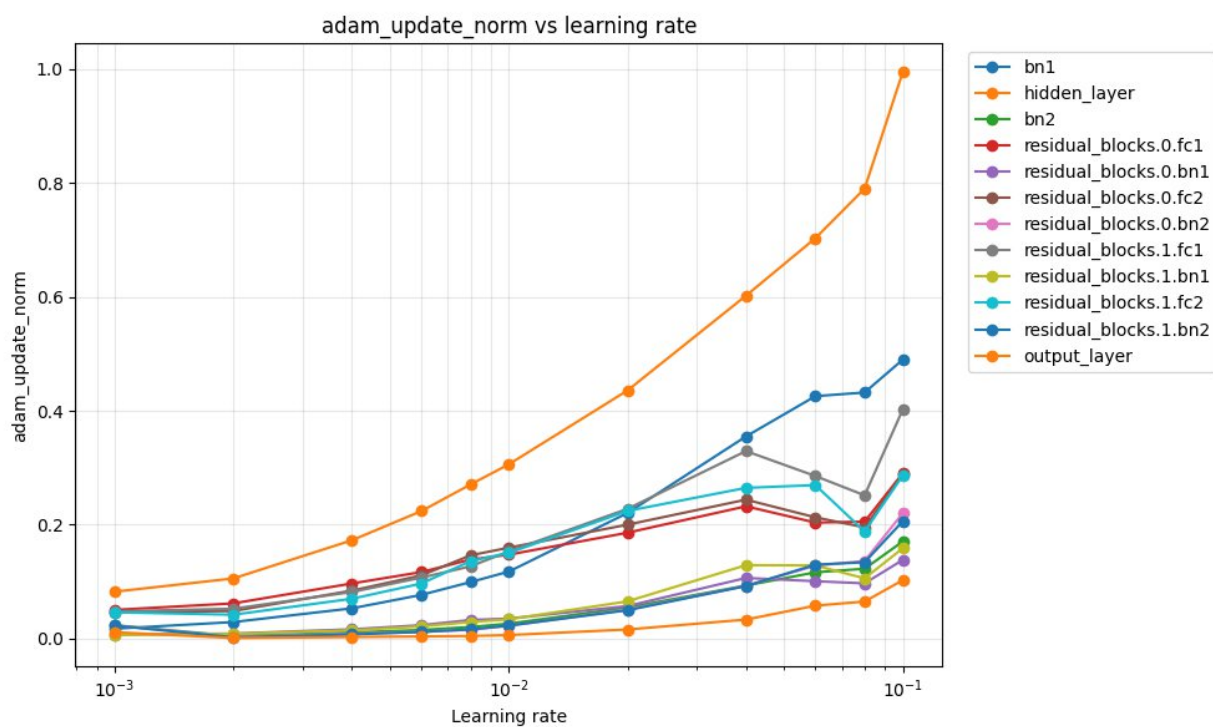
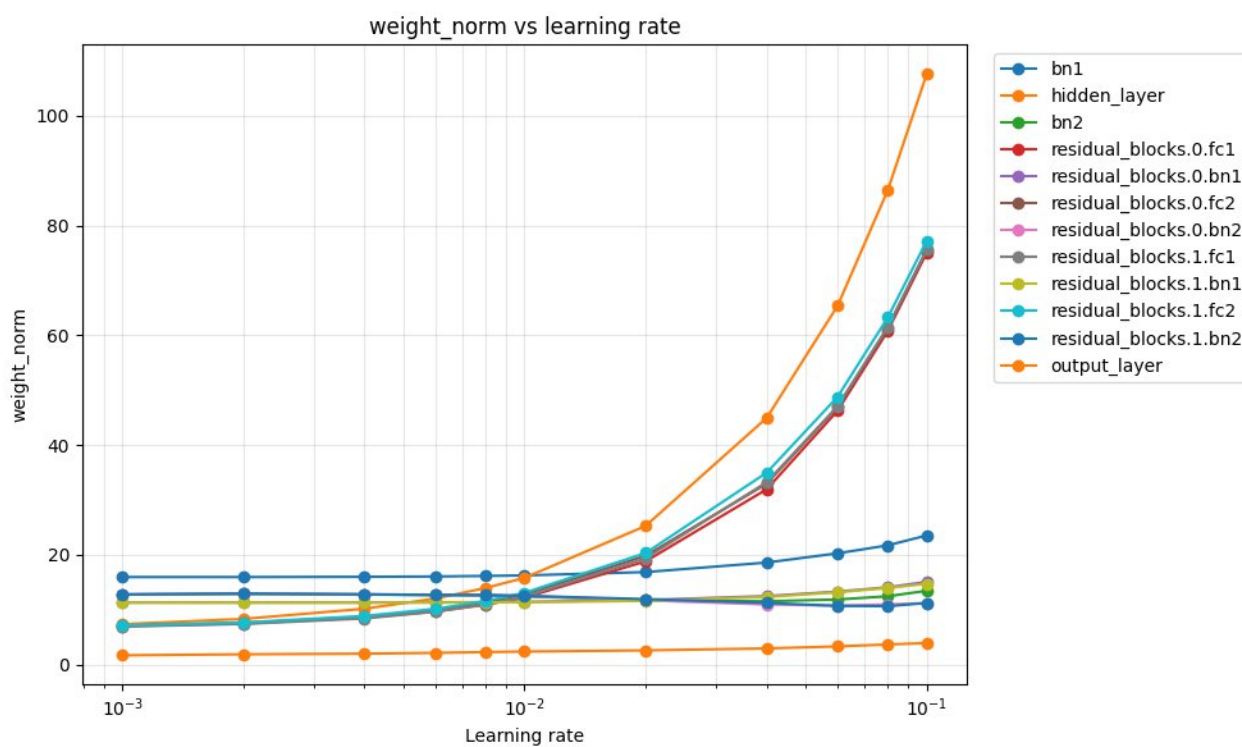
Выше 2 графика - с и без выходным слоем. Видно, что линейные и батчнорм слои имеют разную динамику.



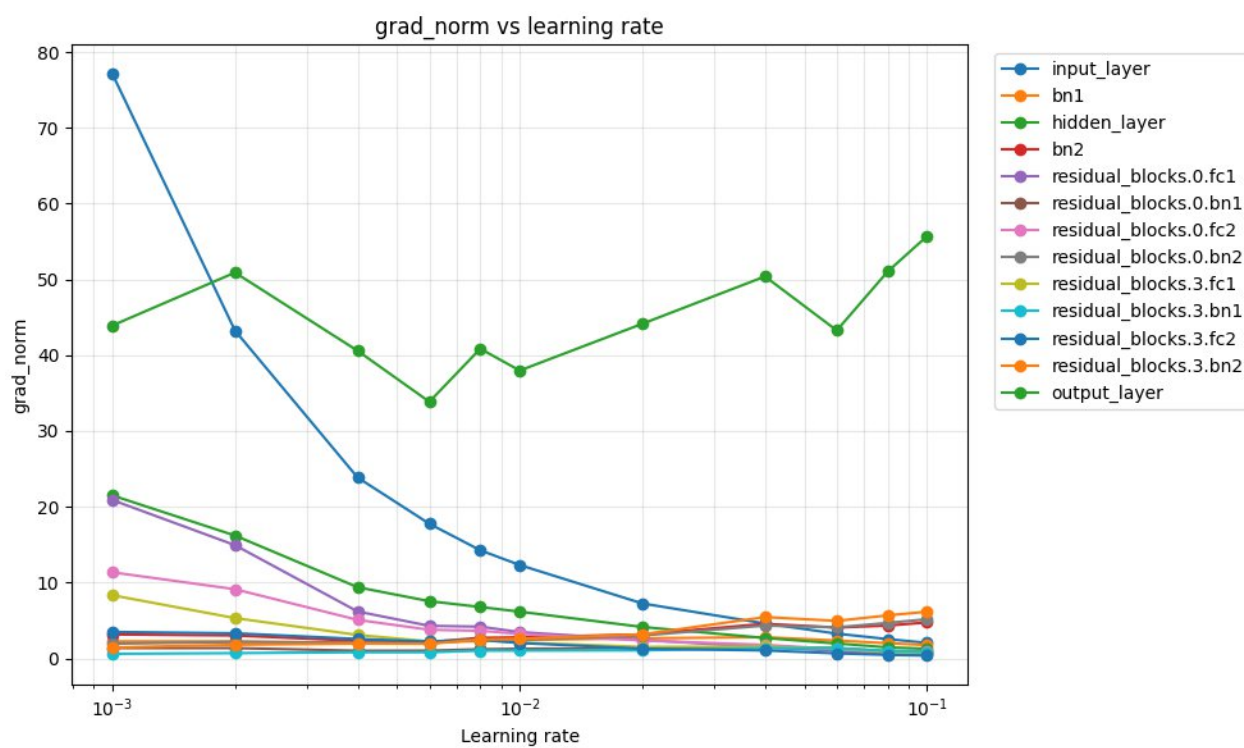
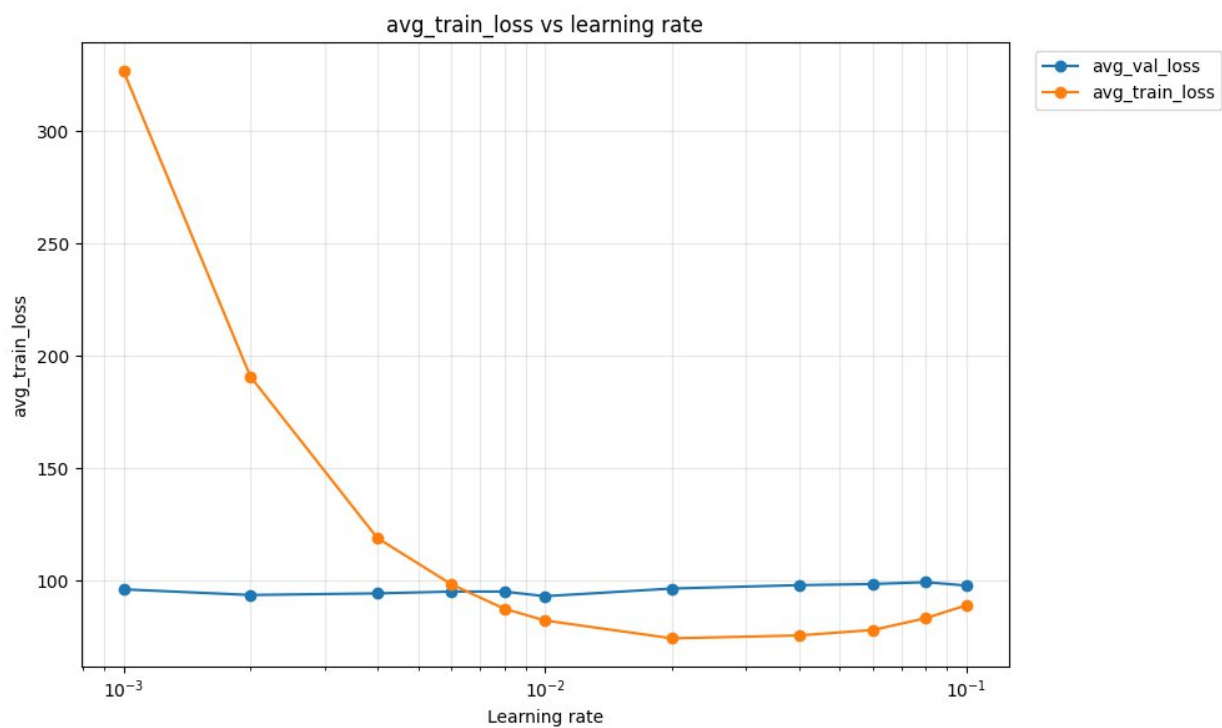
Из последнего рисунка особенно отчетливо видно, что линейные и батчнорм слои обновляются оптимайзером с разной динамикой.

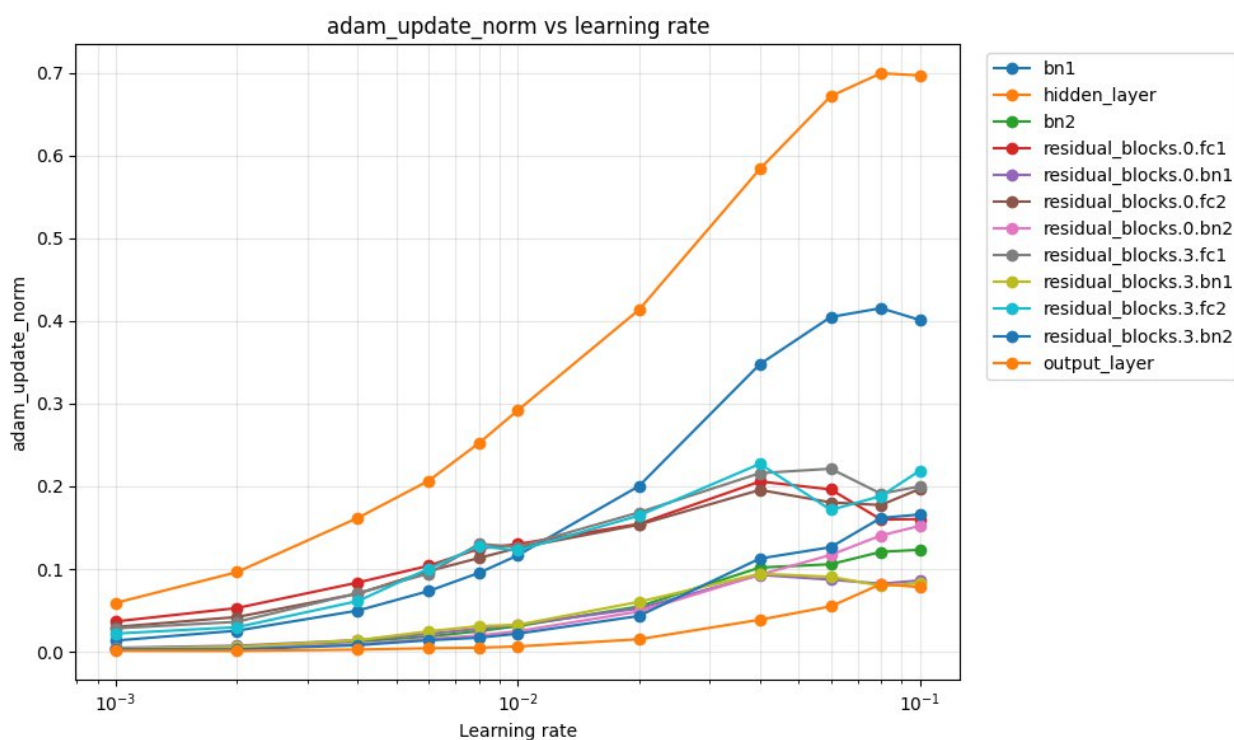
Nrd=2





NRD=4





Видно, что увеличение числа блоков смещает «интервал оптимальности» в сторону больших LR.

ВЫВОДЫ

Лучшая эпоха. Если лучшая и пороговая эпохи находятся рядом, то лучшая эпоха даст результаты лучше. Лосс, который влияет на %solv - это val_loss. Не получалось получить %solv > 90 для val_loss > 57.7 (256-128x1). Но лучшая эпоха может лежать довольно далеко от пороговой эпохи - между ними могут быть десятки запусков. Т.к. чекпоинты будут получены на разных по размеру тренировочных датасетах (лишь частично совпадающих) - чекпоинты будут относиться к существенно отличным обученным моделям - даже близкие значения лосса будут давать разные результаты beam_search. Кроме того, %solv определяется не только значением лосса, но и размером тренировочного датасета (числом эпох). Если тренировочный датасет был слишком мал, то возрастает шанс провала beam_search (совсем низкий %solv при превосходном значении лосса).

Переключение LR уменьшает val_loss и увеличивает train_loss. Значение LR определяет параметрическое пространство обученной модели. Одинаковые значения лосса полученные при разных LR дают разные результаты beam_search. При больших LR модель находит хорошие решения, но не может их исследовать. При малых LR - модель хорошо исследует решения, но эти решения представляют собой локальные минимумы. Поэтому переключение LR дает столь замечательный результат. Сначала получаем хорошие обобщения для задачи, а потом файнтюнинг. Хороший schedule - это здоровые кривые обучения по разным метрикам - без режима инфляции весов и признаков переобучения.

Есть оптимальная эпоха для переключения: переключиться раньше - не дообучится, переключиться позже - «потерять время», впустую потраченные эпохи не дают бонуса при переключении LR и не улучшают результат beam search. При этом, если модель долго обучается в режиме инфляции весов, это приводит к ухудшению результатов beam search.

Есть оптимальная кривая обучения с общим LR.

Слои учатся в своем темпе - если подобрать так, чтобы adam апдейт и веса не демонстрировали инфляцию - что будет? - слои сильно зависят друг от друга, такой подбор требует автоматизации контроля метрик, т.е. дополнительного исследования.

В репозитории приложен ноутбук с кодом и примерами - возможность тренировки слоев с разными скоростями. Предварительно внятных результатов по этим экспериментам не получено, запуски 283 beam_search оказался неудачным, но код и результаты можно посмотреть.

Общая таблица результатов запусков находится рядом с этим ноутбуком на гит хабе.